University of Warsaw
Faculty of Philosophy

Adam Joachim Wełnicki
Record book number: 453676

Punctual presentations of abelian groups

Bachelor's thesis
in the field of Cognitive Science

Thesis written under the supervision of
Dr Michał Wrocławski
Faculty of Philosophy University of Warsaw

Warsaw, November 2025

Abstract

This thesis investigates effective presentations of groups with a particular focus on punctual presentations. The framework of punctual structures, developed in [20], is used to analyze properties of structures computable without delay and has been extensively studied in recent years. A structure over a finite signature is punctual if its domain is the set of natural numbers (or an initial segment thereof) and all functions and relations in its signature are primitive recursive.

To provide better context for the main results, the thesis begins by describing properties of abelian groups in two important smaller subclasses of computable structures: automatic sand polynomial-time (PTIME) structures. Then the core results are presented that concern punctual presentability of two major classes of abelian groups. First, it is shown that every computable torsion-free abelian group admits a punctual presentation. In contrast, the thesis demonstrates that not all computable torsion abelian groups are punctually presentable. The proofs were originally published in [20] but only in a sketchy way. Here the proofs are presented with all detail and a mistake in the proof of the latter theorem is corrected.

Together, these results exemplify both the expressive power and inherent limitations of punctual presentability.

Key words

fully primitive recursive structures, punctual structures, torsion-free groups, torsion groups, punctually robust classes

Title of the thesis in the Polish language

Punktualne prezentacje grup abelowych

Contents

1. Introduction . . . 3
- 1.1. Concept of computability . . . 3
- 1.2. Computable presentations of structures . . . 4
- 1.3. Overview of the thesis . . . 5

2. Preliminaries . . . 7
- 2.1. Standard set-theoretic notation . . . 7
- 2.2. Computability theory . . . 7
  - 2.2.1. Encoding hereditarily finite objects with natural numbers . . . 9
  - 2.2.2. Computable and computably enumerable Sets . . . 10
  - 2.2.3. Characterization of primitive recursive functions . . . 12
- 2.3. Computable structure theory . . . 13
- 2.4. Introduction to diagonal and priority constructions . . . 15
- 2.5. Group theory . . . 17

3. Important subclasses of computable structures . . . 20
- 3.1. Automatic structures . . . 20
- 3.2. PTIME structures . . . 28
  - 3.2.1. Examples of algebraic structures that admit PTIME copy . . . 29
  - 3.2.2. Comparison with automatic structures . . . 30
  - 3.2.3. PTIME groups . . . 30
- 3.3. Punctual structures . . . 31
  - 3.3.1. Punctually robust classes of structures . . . 33

4. Punctual presentability of torsion and torsion-free abelian groups . . . 35
- 4.1. Torsion-free abelian groups . . . 35
  - 4.1.1. Proof: The class of torsion-free abelian groups is punctually robust . 37
- 4.2. Torsion abelian groups . . . 41
  - 4.2.1. Proof: The class of torsion abelian groups is not punctually robust . 43

5. Conclusions . . . 48

Chapter 1

Introduction

1.1. Concept of computability

Computability theory is concerned with the fundamental question of what can, in principle, be computed by an algorithm. Formally, it studies functions on the natural numbers (or other countable domains) and asks whether there exists an effective procedure that, given any valid input, produces the correct output after a finite number of steps. An "effective procedure" here is intended to capture the informal notion of a method that is precise, mechanical, and guaranteed to terminate with the correct result whenever the function is defined on the input. This line of inquiry was initiated in the 1930s by Church, Turing, and others, leading to the formulation of equivalent models of computation (e.g. the λ-calculus [3], Turing machines [34], general recursive functions [24, 34]) that describe the same class of effectively computable functions. In the preliminary section, we will define general recursive functions formally. If a function can be described using one of these models, it can be computed using any of them, and such a function is called computable. More precisely, *total computable* if it is defined for every input and *partial computable* if it may not halt on some inputs. This equivalence gives strong evidence that these models capture what we intuitively mean by computable by an algorithmic procedure. Therefore, it is a standard practice to assume the ***Church-Turing thesis*** which asserts:

Every function that can be computed by any effective, mechanical method can also be computed by a Turing machine (and thus by any equivalent formal model of computation).

While this is not a theorem that can be proved, because "effective method" is an intuitive, informal notion, it is universally accepted as the correct characterization of what it means for a function to be computable.

While computability theory describes what can be computed by algorithmic methods, this generality often comes at the cost of unpredictability. Computable functions may fail to halt on certain inputs (when they are only partial), and total computable functions can require arbitrarily large and uncontrolled computational resources. This unpredictability is tied to **unbounded search**: many general methods for defining computable functions involve searching for the least value satisfying some condition, with no bound on how long the search may last, i.e *unbounded loops*. Such procedures, formalized by the minimization μ operator (2.3), may never terminate if no suitable value exists, and even when they do, there is no upper bound on the computational effort required. Thus, it is useful to study ways of restricting the class of computable functions to some subclass of them.

One of the most natural and well-studied subclasses of computable functions is the class of *primitive recursive functions*. We will define those functions formally in the section 2.2.3, however, stated informally, these functions are built from basic initial functions (zero, successor, projections) using composition and primitive recursion, a process that defines each value of function in terms value of the same function with a smaller arguments or a simpler function, in a completely bounded way. Because primitive recursion always involves a predetermined number of computation steps for each input, these functions avoid unbounded search entirely. As a result, every primitive recursive function is total and its evaluation is fully predictable. Moreover, most functions that arise in typical mathematical and computational contexts, such as multiplication, factorial, and exponentiation can be expressed using primitive recursion, making this class both natural and widely applicable.

This restriction motivates a reformulation of the Church-Turing thesis for specifically primitive recursive functions. Restricting "any effective methods" to only those methods that are *bounded*, i.e. do not use unbounded search and always produce an output after a finite number of steps. Hence, we will assume to be true the *Restricted Church-Turing thesis for primitive recursive functions* [20], which asserts:

If we can compute a function with an algorithm without using unbounded loops (such as WHILE . . . DO, REPEAT . . . UNTIL, and GOTO in a Pascal-like language), then this function is primitive recursive.

Another, more formal way of formulating a similar idea, in the langue of computer science, is using the definition of loop programs [28] and theorem 1.1, which equates primitive recursive functions with those computable using loop programs.

Theorem 1.1. *The functions computable by Loop programs are precisely the primitive recursive functions.[28]*

However, other more restrictive approach to computability focuses not only on halting and boundedness. For instance, *automata theory* studies classes of functions or languages recognized by abstract machines with restricted memory, such as finite automata, for further reading on finite automat, we refer the reader to [18]. These models capture notions of computation that are inherently more constrained than Turing machines - finite automata can only recognize regular languages and have no unbounded memory. Another common restriction is based on *computational complexity*, where we classify functions according to the time or space required by an algorithm to produce an output. A prominent example is the case of **polynomial-time computable functions** (PTIME), which are functions computable by a deterministic Turing machine in time bounded by a polynomial in the input size. These restrictions provide a framework for studying efficient or feasible computation, contrasting with the general notion of computability, which may allow arbitrarily long computations. Together with primitive recursion, these perspectives illustrate the spectrum of methods for controlling and understanding the practical behavior of algorithms. We will discuss automata and PTIME more formally in Chapter 3.

1.2. Computable presentations of structures

Building on the notion of computable functions in 1960s Mal'cev [26] and Rabin [33] interdependently introduced definition of algorithmically presented algebraic structures, thus creating new area of study in computability theory - *computable structure theory*. Computable structure theory extends ideas from computability theory to the study of mathematical structures in which the operations, relations, and elements can themselves

be effectively computed. Instead of focusing solely on individual functions, computable structure theory considers entire algebraic structures, such as groups, rings, graphs, and fields. This framework allows us to ask questions such as: which properties of a structure can be effectively decided? Can an isomorphism between two structures be effectively constructed? Important to us, what properties do different **computable presentations** of a structure have?

Analogous to computability theory, computable structure theory allows us to reason about computable structures; it does not impose restrictions on the computational resources required to compute answers to questions about its existential diagram. As a result, even total computable structures can involve operations whose evaluation is unpredictable or requires arbitrarily large computational resources. To gain finer control over this computation, one can restrict computation models used for computing said structures. There are several natural ways to impose restrictions on computations in computable structures, we will consider the approaches discussed above. First, one can consider *automata-based structures*, where the domain and operations of the structure are constrained to those recognizable by finite automata. However, this imposes limits on the complexity of structures that we will find to be too restrictive, making the class of structures computed too small. We can study *polynomial-time (PTIME) structures*, which have been shown by Cenzer and Remmel [9] to admit more complex structures in their foundational work on PTIME structures. PTIME structures are characterized by operations and relations being computable within polynomial time with respect to the input size. By Cobham–Edmonds thesis [19, 11], restricting the class of structures to only those computable in polynomial time would ensures feasible computation, but one can argue the restriction to polynomial-time to be too arbitrary. It is more natural for us to focus on *fully primitive recursive structures*, which we will call **punctual**, where all operations, relations are defined using only primitive recursion and the domain is the set of all natural numbers and in the finite case some initial segment of it. This ensures that all computations within the structure are total and predictable, while not making the class of structures too small. On the other hand, if we allow the domain of a structure to be an arbitrary primitive recursive subset of natural numbers, thus restricting ourselves only to primitive recursive (not *fully* primitive recursive) structures, this would not impose sufficient limitations as demonstrated by Theorem 3.27.

After introducing the concept of punctual structures and motivating their role as a natural restriction on computability of structures, this thesis focuses on presenting two results concerning specific classes of algebraic structures: torsion-free abelian groups and torsion abelian groups. These results, originally established in [4], are reformulated here to provide more detailed and accessible proofs and correct a mistake in the original proof. The first result establishes that every computable torsion-free abelian group admits a punctual presentation. In contrast, the second result shows that not all computable torsion abelian groups can be presented in this way, highlighting the limitations of punctuality in certain algebraic contexts.

1.3. Overview of the thesis

This thesis is structured as follows.

Chapter 2 provides all necessary preliminaries. We begin with a brief review of standard set-theoretic notation and then introduce key notions from computability theory, including general recursive functions, computably enumerable sets, and the characterization of primitive recursive functions. Special attention is given to the distinction between

general computability and primitive recursion, since this distinction later becomes central in defining punctual structures. The chapter then introduces foundational concepts from computable structure theory, such as computable presentations and computable isomorphisms, followed by an overview of diagonal and priority constructions. We conclude the chapter with basic group theoretic definitions, particularly those related to torsion, torsion-free groups, and direct sums, since these concepts are the main focus of later chapters.

Chapter 3 examines several important subclasses of computable structures. We begin with automatic structures, the most restrictive class considered, and describe why finite automata severely constrain the algebraic structures that can be automatically presented. We then discuss PTIME structures, which use polynomial-time computational restrictions to obtain more expressive, but still feasibly computable, presentations. After that, we introduce punctual structures, which restrict computation to primitive recursive functions. This class turns out to be significantly more expressive than automatic structures while retaining strong boundedness properties. The chapter also highlights computational advantages and limitations of each subclass, preparing the ground for the study of punctual presentability of abelian groups.

Chapter 4 contains the core results of the thesis. We analyze the punctual presentability of two major subclasses of abelian groups. First, we consider torsion-free abelian groups and show that every computable torsion-free abelian group admits a punctual presentation. The proof adapts the argument from [20], while providing much more detail compared to a more sketchy original presentation of the proof. Next, we turn to torsion abelian groups and show that, unlike the torsion-free case, not all computable torsion groups can be presented punctually. Again, we fill in all the missing details in the proof and additionally we fix a mistake regarding an incorrect approximation of certain parameter in the original construction. Together, these results illustrate both the capabilities and the limitations of punctual structures in computable algebra.

Chapter 5 contains the Conclusions. This chapter summarises the main results of the thesis, places them in the broader context of subclasses of computable structures and describes further directions.

Chapter 2

Preliminaries

In this chapter we will introduce definitions and theorems from various areas that will be useful later in the thesis.

2.1. Standard set-theoretic notation

We denote the set of natural numbers $\{0, 1, 2, 3, \dots\}$ by $\mathbb{N}$ or ω. Sequences will be indexed using subscripts. For any set A, we write A^n for the set of all n-tuples $(a_1, a_2, \dots, a_n)$ with $a_i \in A$ for each $i = 1, \dots, n$. By $A^{<\omega}$ we denote the set of all finite tuples of the form $(a_1, a_2, \dots, a_n)$, where each $a_i \in A$ and $n \in \mathbb{N}$. If f is a function we will write $f^k(x)$ to denote applying function f to x k-times.

2.2. Computability theory

Introduction to computability

Here we introduce necessary concepts from computability theory, that in the next section 2.3 we will be applied to algebraic structures. For more in depth introduction to computability theory we recommend [34].

In computability theory, a function is said to be **computable** if it can be calculated by effective methods. These "effective methods" can be understood as code written in any programming language that calculates value of a function for a given input, be it Python, Java, or C, or it can be thought of as more abstract model of computation e.g Turing machine as long as it is purely mechanical and returns answer in finite time. We assume Church-Turing thesis 1.1. Models of computation such as Turing machines and programming language implementations are equivalent in their computational power. Thus we define computable functions:

Definition 2.1. A *partial computable function* is a function $f : \mathbb{N}^k \to \mathbb{N}$ for some $k \geq 1$ such that there exists an effective algorithm which, given any input $(n_1, \dots, n_k) \in \mathbb{N}^k$, halts and outputs $f(n_1, \dots, n_k)$ whenever f is defined. If f is not defined on some input, the machine will not halt.

Definition 2.2. A *total computable function* is a partial computable function $f : \mathbb{N}^k \to \mathbb{N}$ that is defined for all inputs in $\mathbb{N}^k$, i.e., the corresponding computation halts on every input.

Note that, in standard mathematics, a function is usually defined as a mapping between sets. That is, if $f : A \to B$ is a function, then for every $a \in A$ there exists exactly one $b \in B$ such that $f(a) = b$. Functions are, by definition, total: they assign a value to every element of their domain. When a formula appears to be undefined for some inputs, the usual mathematical approach is to restrict the domain so that the function remains total. For example, the expression

$$g(x) = \frac{1}{x}$$

is not a function from $\mathbb{R}$ to $\mathbb{R}$, because it is not defined at $x = 0$. Instead, in standard mathematics, it is formally considered a function

$$g : \mathbb{R} \setminus \{0\} \to \mathbb{R},$$

with the point $x = 0$ excluded from the domain in order to preserve totality.

In contrast in computability theory, rather than always insisting that functions be total, we allow for the possibility of *partial functions*. These are functions whose values may be undefined for some inputs. Importantly, the domain is usually fixed as $\mathbb{N}$ (or $\mathbb{N}^k$ for k-ary functions), but the function may not yield an output in some cases. This mirrors the behavior of algorithms, a computation may halt and produce an output on some inputs, while on other inputs it may never halt, leaving the output undefined.

General recursive functions

Below we are going to define *general recursive functions*. This will provide us with a precise mathematical framework for defining unbounded search and, as we will see in the section on primitive recursive functions (2.2.3), allow us to formally differentiate between functions that use unbounded search and primitive recursive ones. While we present this formal definition, throughout the thesis we will abstract away from the formalism and think in terms of well defined algorithms, when discussing computability.

Definition 2.3. A function $f : \mathbb{N}^n \to \mathbb{N}$ is general recursive if f can be generated from the basic functions $s(x) = x + 1$, $o(x) = 0$, $I^n_m(x_1, \ldots, x_n) = x_m$, by composition, the primitive recursion operator $h = \mathcal{P}(f, g)$:

$$\begin{aligned} h(x_1, \ldots, x_n, 0) &= f(x_1, \ldots, x_n) \\ h(x_1, \ldots, x_n, s(y)) &= g(x_1, \ldots, x_n, y, h(x_1, \ldots, x_n, y)) \end{aligned}$$

and the *minimization operator* $\mu(f)$, that given a function $f(x_1, \ldots, x_n, y)$:

$$\mu(f)(x_1, \ldots, x_n) = z \iff f(x_1, \ldots, x_n, i) > 0 \quad \text{for } i = 1, \ldots, z-1 \tag{2.1}$$

$$f(x_1, \ldots, x_n, z) = 0 \tag{2.2}$$

The primitive recursive operator will be discussed in detail in Section 2.2.3. Here, however, we characterize the minimization operator (or μ-operator). This operator represents unbounded search: it seeks the least value z for which the function f returns 0, without any a priori bound on the number of steps required. Even assuming that f is defined for every smaller argument, we cannot always determine in advance when the computation will halt. In cases where no such z exists for some input, the function is undefined at that input and is defined as **partial computable**. If, on the other hand, such a z exists for every input $(x_1, \ldots, x_n)$, the function is total computable. Informally, this corresponds to statements like "search for x" or "wait until x produces y."

2.2.1. Encoding hereditarily finite objects with natural numbers

By hereditarily finite, we mean objects $x \in \mathrm{HF}$, where:

$$\mathrm{HF} = \{x \mid x \text{ is finite and every element of } x \text{ is in HF}\}$$

Examples:

$$\begin{aligned}
&\varnothing \in \mathrm{HF},\\
&\{\{\varnothing\}\} \in \mathrm{HF},\\
&\{\varnothing, \{\varnothing\}\} \in \mathrm{HF},\\
&\{\mathbb{N}\} \notin \mathrm{HF} \quad \text{(since } \mathbb{N} \text{ is infinite).}
\end{aligned}$$

In computable structure theory the domain is always countable, therefore we can assign each object in domain a unique natural number. Consequently, without loss of generality, we can choose the domain of the functions to be the set of natural numbers or its subset. This domain is more convenient and every tuple (set of sets of natural numbers) can be effectively encoded as a natural number and thus every countable object can:

Remark 2.4. Any set of finite set of natural numbers and hereditary finite set of sets of natural numbers, can be encoded by a single natural number. Consequently, every finite tuple of natural numbers can also be encoded as a natural number.

Proof. We show this by constructing such an encoding.

1. **Encoding finite tuples as sets.**
   Any finite tuple can be turned into a finite set of sets. For example, a pair (a, b) can be represented as
   $$(a, b) \longmapsto \{\, a,\ \{a, b\}\,\}.$$
   If we can encode finite sets as numbers, then we can also encode finite tuples as numbers.

2. **Encoding sets as binary sequences.**
   Let $S \subset \mathbb{N}$ and S be finite. Define its binary encoding as the infinite sequence $(b_0, b_1, b_2, \dots)$ given by
   $$b_n \;=\; \begin{cases} 1 & \text{if } n \in S, \\ 0 & \text{if } n \notin S \end{cases} \qquad \text{for all } n \in \mathbb{N}.$$
   For example, if $S = \{0, 2, 4\}$, its encoding is
   $$(1, 0, 1, 0, 1).$$
   Any finite binary sequence can itself be read as a (possibly very large) natural number by interpreting it as a binary numeral. For example, the sequence $(1, 0, 1)$ corresponds to
   $$1 \cdot 2^0 + 0 \cdot 2^1 + 1 \cdot 2^2 \;=\; 5.$$

3. **Encoding sets of sets of natural numbers.**
   Now suppose we have $S = \{S_0, S_1, S_2, \dots, S_n\}$, a finite set of finite subsets of $\mathbb{N}$. Each S_i is first turned into a binary sequence $b^{(i)}$. We then combine these sequences into one single binary sequence by *concatenation*, then we can repeart the above method to encode the set $\{b^{(0)}, \dots, b^{(n)}\}$, note that this method does not allow us to mix objects from different levels (e.g sets of numbers and sets of sets of numbers), but an easy refinement of this method could solve this problem.

This concludes our construction of encoding of HF sets of natural numbers. $\square$

2.2.2. Computable and computably enumerable Sets

Let:

$$\varphi_1, \dots, \varphi_k, \dots$$

be an effective enumeration of of all partial computable functions indexed by natural numbers, meaning we can mechanical go from index e to a description of algorithm that computes φ_e and vice versa. We use $\varphi_e(n)$ to denote the output of eth function on input n.

Definition 2.5. A *computation step* is a single operation of a given computational model (e.g., a Turing machine or program).

Formally, it corresponds to the smallest indivisible action of a machine: reading or writing a symbol, moving the tape. For more abstract algorithms and for our purpose, we treat a step as a single line of execution or a single instruction. If the function returns an answer after a finite number of steps, we write $\varphi_e(n) \downarrow$ analogously, we write $\varphi_e(n) \uparrow$ if the function diverges on the answer (never returns an answer). We write $\varphi_{e,s}(n)$ to express output of $\varphi_e(n)$ after s steps.

Definition 2.6. A *characteristic function* of a set $A \subseteq \mathbb{N}^k$ is a function $\chi_A : \mathbb{N}^k \to \{0, 1\}$ such that for $(n_1, \dots, n_k) \in \mathbb{N}^k$:

$$\chi_A(n_1, \dots, n_k) = 1 \iff (n_1, \dots, n_k) \in A$$

and

$$\chi_A(n_1, \dots, n_k) = 0 \iff (n_1, \dots, n_k) \notin A$$

Definition 2.7. A set is *computable* if and only if its characteristic function is computable.

So, a set is computable if and only if there exists such a function, called its characteristic function, whose computation halts for every input and returns 0 when the input is not in this set and 1 when it is an element of this set. Hence a set is computable if we can determine whether an number is or is not in the set by using a algorithm.

Definition 2.8. A set $A \subseteq \mathbb{N}^k$ is called *computably enumerable* (c.e) if there exists a partial computable function

$$\varphi : \mathbb{N}^k \to \{0, 1\}$$

such that

$$\varphi(n_1, \dots, n_k) = 1 \iff (n_1, \dots, n_k) \in A.$$

Equivalently,

Definition 2.9. A is **computably enumerable (c.e.)** if either $A = \emptyset$ or there exists a total computable function

$$g : \mathbb{N} \to A$$

that is surjective. Such a map g is called an *effective enumeration* of A, meaning that we can effectively list all elements of A:

$$A = \{g(0), g(1), g(2), \dots\}.$$

Often, for brevity, we use a subscript notation and write $A = \{a_0, a_1, a_2, \dots\}$ where $a_n = g(n)$.

Less rigorously we can state that if we have effective procedure for listing all the elements of a set A, A is c.e.

Remark 2.10. Definitions 2.8 and 2.9 are equivalent.

Proof. We prove both implications.

($\Rightarrow$) If there is a partial computable function $\varphi_e : \mathbb{N} \to \{0, 1\}$ such that $\varphi(x) = 1 \iff x \in A$, then either $A = \emptyset$ there exists an effective enumeration $g : \mathbb{N} \to A$.

Assume there is a partial computable function $\varphi_e : \mathbb{N} \to \{0, 1\}$ such that

$$\varphi(x) = 1 \iff x \in A.$$

If $A = \emptyset$, then the conclusion follows trivially otherwise fix any $a \in A$ (this is necessary for finite case). We construct a total computable surjection g that enumerates A.

Construction of g at stage s:

1. Compute $\varphi_{e,s}(1), \varphi_{e,s}(2), \ldots, \varphi_{e,s}(s)$

2. Whenever for some input $x \leq s$, $\varphi_e(x)$ halts and outputs 1 and x has not yet been listed, assign it least natural number n not yet assigned by setting $g(n) = x$, otherwise assign $g(n) = a$.

3. Continue indefinitely. Every element of A will eventually appear and receive a unique index.

Because this process lists every element of A, g is total and onto. Thus g is an effective enumeration of A. Note that if A is infinite, we could skip repeatedly adding a to the sequence and as a result we would obtain a bijective sequence

Remark 2.11. This method of parallel simulation is called **dovetailing**. It interleaves the steps of infinitely many computations so that no single computation is ignored indefinitely, even if some never halt. We will be implicitly using it any time we say we have **enumeration** or we say *"wait until X shows in Y"*.

($\Leftarrow$) If $A = \emptyset$ or there exists an effective enumeration g, then there is a partial computable function $\varphi_e : \mathbb{N} \to \{0, 1\}$ such that $\varphi(x) = 1 \iff x \in A$.

If $A = \emptyset$, we set $\varphi(x) = 0$, for all $x \in \mathbb{N}$, now assume there is a total computable bijection $g : \mathbb{N} \to A$. We construct a partial computable function φ with

$$\varphi(x) = 1 \iff x \in A.$$

Construction of φ:

1. On input x, compute $g(0), g(1), g(2), \ldots$ sequentially.

2. If some s satisfies $g(s) = x$, halt and output 1.

3. If $x \notin A$, this search never terminates, so $\varphi(x)$ is undefined.

This function φ is partial computable with domain exactly A, proving that A is c.e.

$\square$

2.2.3. Characterization of primitive recursive functions

Definition 2.12. A function $f : \mathbb{N}^n \to \mathbb{N}$ is primitive recursive (p.r) if f can be generated from the basic functions $s(x) = x + 1,\ o(x) = 0,\ I_m^n(x_1, \ldots, x_n) = x_m$, by composition and the primitive recursion operator $h = \mathcal{P}(f, g)$:

$$h(x_1, \ldots, x_n, 0) = f(x_1, \ldots, x_n),$$
$$h(x_1, \ldots, x_n, s(y)) = g(x_1, \ldots, x_n, y, h(x_1, \ldots, x_n, y))$$

This is very similar to the definition of general recursive functions (Definition 2.3), the only difference being the exclusion of the minimization operator, which is responsible for unbounded search. In the absence of the minimization operator or any construct such as unbounded loops, any effectively computable equivalent function obtained this way is *primitive recursive*.

Accordingly, we may abstract from the formal definition and adopt the following intuitive perspective: any algorithm whose instructions include operations of the form "after s steps, perform X," where X is itself a primitively recursive instruction and does not involve instructions of the form "wait until X," is primitive recursive. Here, the parameter s need not be constant; it may depend on the input in a primitive recursive way.

Another important observation we can make by compering definitions 2.2.3 and 2.3 is that **all primitive recursive functions are total**. Recall that general recursive functions are not total when minimization operator never finds z for which $f(x_1, \ldots, x_n, z) = 0$, but primitive recursive functions do not have minimization operator and are therefore *total*.

To better understand why this abstraction is correct, let us analyze the definition of the primitive recursive operator $h = \mathcal{P}(f, g)$. The first line defines the base case using parameters $(x_1, \ldots, x_n)$. The second line defines the value of h at step $s(y)$, given the fixed parameters $(x_1, \ldots, x_n)$ and the previous value $h(x_1, \ldots, x_n, y)$. This operator effectively acts as a bounded "for loop": to compute $h(x_1, \ldots, x_n, y)$, we recursively compute $h(x_1, \ldots, x_n, y - 1)$ and so forth, until we reach the base case $h(x_1, \ldots, x_n, 0) = f(x_1, \ldots, x_n)$. Therefore, as with general computability, we do not need to concern ourselves with the low-level mechanics of the computation, as long as it does not involve unbounded loops.

Nevertheless, it is still worthwhile to demonstrate how this formal, low-level definition can be applied to construct a function. A simple but instructive example is the definition of addition as a primitive recursive function.

Example 2.13. To do this, we recreate the following definition of addition in terms of what is allowed in the definition of primitive recursive functions:

$$a + 0 = a$$
$$s(a) + b = s(a + b)$$

To obtain the first equation, we define the initial step of our recursive operator h, as $h(y, 0) = I_1^1(y)$. Now to obtain the "body" of the loop, we need to define g from the definition 2.12, $h(x, s(y))$, the left side of the definition equation, and the left side of the definition of addition. We can see that $h(x, y)$ is equivalent to $x + y$. To obtain the right side, we define the body of the loop as $g = s(I_3^3(x, y, h(x, y)))$ follows that:

$$h(x, s(y)) = g(x, y, h(x, y)) = s(I_3^3(x, y, h(x, y))) = s(h(x, y))$$

Substituting $h(x, y)$ for $x + y$ for clarity, we get:

$$x + s(y) = g(x, y, x + y) = s(I_3^3(x, y, x + y)) = s(x + y)$$

This gives us the second part of our addition definition.

Below we prove a theorem, that will be very useful later in this thesis

Theorem 2.14. *There exists an effective enumeration of all primitive recursive functions*

We can represent primitive recursive functions as derivations constructed using sequence of components listed in the definition 2.12.

Proof. To get this enumeration, we need to construct every primitive recursive function, so we need to systematically create functions out of the five basic components listed in definition 2.12. This construction is going to be carried out in infinitely many stages and in the limit will give an enumeration of all primitive recursive functions. Each stage of construction is indexed with natural number.

At stage $s = 0$ of construction, we start with functions $s(x), o(x), I_1^1(x)$. At the beginning of stage s of construction, we have the current enumeration of primitive recursive functions:

$$p_1, \ldots, p_i$$

During a stage s, we first add all the projection functions of the form

$$I_1^{s+1}(x_1, \ldots, x_{s+1}), \ldots, I_{s+1}^{s+1}(x_1, \ldots, x_{s+1})$$

to our enumeration, yielding an enumeration:

$$p_1, \ldots, p_j$$

where $j > i$. Finally, we add all possible compositions of those functions and apply the primitive recursive operator $h = \mathcal{P}(f, g)$ to every pair of functions in the current enumeration and add each resulting functions to the enumeration. This concludes stages s of the construction.

At the end of the stage s we obtain a finite sequence:

$$p_1, p_2, \ldots, p_{k_s}$$

The enumeration of all primitive recursive functions is a set-theoretic sum of all these finite sequences.

□

Note that each such derivation can easily be used to construct an algorithm for a given function.

Definition 2.15. A set is said to be *primitive recursive* if its characteristic function is primitive recursive.

This definition of primitive recursive sets parallels the definition of computable sets (see Definition 2.7).

2.3. Computable structure theory

In this section we introduce some basic concepts from Computable structure theory.

The following definitions come from [35].

Definition 2.16. A *structure* is a tuple

$$\mathcal{A} = \big(A, (R_i^{\mathcal{A}})_{i\in I}, (f_j^{\mathcal{A}})_{j\in J}, (c_k^{\mathcal{A}})_{k\in K}\big)$$

where A is a nonempty set called the *domain* of the structure, each $R_i^{\mathcal{A}} \subseteq A^{n_i}$ with $i \in I$ is an n_i-ary relation on A, each $f_j^{\mathcal{A}} : A^{m_j} \to A$ with $j \in J$ is an m_j-ary function on A, and each $c_k^{\mathcal{A}} \in A$ with $k \in K$ is a constant symbol interpreted in A. The collection of relation symbols $\{R_i : i \in I\}$, function symbols $\{f_j : j \in J\}$, and constant symbols $\{c_k : k \in K\}$, together with their respective arities, is called the *signature* of the structure. We assume that I, J, K are pairwise disjoint sets of natural numbers.

Example 2.17. Tuple $\mathcal{N} = (\mathbb{N}, <^{\mathcal{N}}, +^{\mathcal{N}}, \cdot^{\mathcal{N}}, S^{\mathcal{N}}, 0^{\mathcal{N}}, 1^{\mathcal{N}})$ is a *structure*, with:

- **Domain**: $A = \mathbb{N} = \{0, 1, 2, 3, \dots\}$
- **Relations**: One binary relation symbol $<$, interpreted as $<^{\mathcal{N}} \subseteq \mathbb{N} \times \mathbb{N}$
- **Functions**: $+^{\mathcal{N}} : \mathbb{N} \times \mathbb{N} \to \mathbb{N}$, $\cdot^{\mathcal{N}} : \mathbb{N} \times \mathbb{N} \to \mathbb{N}$, $S^{\mathcal{N}}(n) = n + 1$
- **Constants**: $0^{\mathcal{N}} = 0$, $1^{\mathcal{N}} = 1$

Definition 2.18. For a structure

$$\mathcal{A} = \big(A,\ R_1^{\mathcal{A}}, \dots, R_k^{\mathcal{A}},\ f_1^{\mathcal{A}}, \dots, f_m^{\mathcal{A}}\big),$$

where each $R_i^{\mathcal{A}} \subseteq A^{n_i}$ is an n_i-ary relation and each $f_j^{\mathcal{A}} : A^{m_j} \to A$ is an m_j-ary function, and for any $B \subseteq A$, we say that B is *closed under the functions* of $\mathcal{A}$ if for all j and all $b_1, \dots, b_{m_j} \in B$,

$$f_j^{\mathcal{A}}(b_1, \dots, b_{m_j}) \in B.$$

If B is closed under the functions of $\mathcal{A}$, the *induced substructure* on B is defined as

$$\mathcal{A} \restriction B = \Big(B,\ R_1^{\mathcal{A}} \cap B^{n_1}, \dots, R_k^{\mathcal{A}} \cap B^{n_k},\ f_1^{\mathcal{A}}\restriction B^{m_1}, \dots, f_m^{\mathcal{A}}\restriction B^{m_m}\Big),$$

where each $f_j^{\mathcal{A}}\restriction B^{m_j} : B^{m_j} \to B$ is the restriction

$$(b_1, \dots, b_{m_j}) \mapsto f_j^{\mathcal{A}}(b_1, \dots, b_{m_j}).$$

Definition 2.19. Let $\mathcal{A}$ be a structure with domain A and signature

$$\sigma = \{R_0, \dots, R_k, f_0, \dots, f_m, c_1, \dots, c_n\}.$$

We say that the functions, relations and constants of $\mathcal{A}$ are *uniformly computable* if there exists a single algorithm φ such that, for every index i and every tuple $\bar{x} \in A^{<\omega}$,

$$\varphi(i, \bar{x}) = \begin{cases} f_i(\bar{x}) & \text{if } i \in I, \\ 1 & \text{if } i \in J \text{ and } \bar{x} \in R_i, \\ 0 & \text{if } i \in K \text{ and } \bar{x} \notin R_i, \\ c_k^{\mathcal{A}} & \text{if } k \in K \end{cases}$$

Definition 2.20. A structure $\mathcal{A}$ is *computable* if its domain A is computable and the functions, relations and constants of $\mathcal{A}$ are *uniformly* computable.

Example 2.21. A structure $\mathcal{N} = (\mathbb{N}, <, +, \cdot, S, 0, 1)$ from example 2.17 is computable. We can effectively decided if the number is a positive integer, we can algorithmically compute standard addition, successor function and decided standard inequities on natural numbers.

Definition 2.22. Let $\mathcal{A}$ and $\mathcal{B}$ be structures over the same signature σ. We say that $\mathcal{A}$ is *isomorphic* to $\mathcal{B}$, written $\mathcal{A} \cong \mathcal{B}$, if there exists a bijection $f : A \to B$ such that it is a *homomorphism* i.e. the following conditions hold: for every constant symbol $c \in \sigma$, $f(c^{\mathcal{A}}) = c^{\mathcal{B}}$ and for every n-ary function symbol $g \in \sigma$ and all $a_1, \ldots, a_n \in A$,

$$f\big(g^{\mathcal{A}}(a_1, \ldots, a_n)\big) = g^{\mathcal{B}}\big(f(a_1), \ldots, f(a_n)\big)$$

and for every n-ary relation symbol $R \in \sigma$ and all $a_1, \ldots, a_n \in A$,

$$(a_1, \ldots, a_n) \in R^{\mathcal{A}} \iff (f(a_1), \ldots, f(a_n)) \in R^{\mathcal{B}}$$

A bijection $f : A \to B$ satisfying these conditions is called an *isomorphism* from $\mathcal{A}$ to $\mathcal{B}$.

Definition 2.23. Let $\mathcal{A}$ and $\mathcal{B}$, such that $\mathcal{A} \cong \mathcal{B}$ and A is a domain of $\mathcal{A}$ and B is a domain of $\mathcal{B}$, then if $f : A \to B$ is an isomorphism between $\mathcal{A}$ and $\mathcal{B}$ and moreover f is computable, then we say that $\mathcal{A}$ and $\mathcal{B}$ are *computably isomorphic*.

2.4. Introduction to diagonal and priority constructions

In this section we will discuss the idea behind diagonal and priority constructions. This is a commonly used method in computability theory, as well as in some other areas of mathematics, and is used to prove that a mathematical object A exists which does not have a certain property (often computability-related) by ensuring that A is different from each object with this property. A well-known example comes from set theory where we prove that the there is no surjective function from $\mathbb{N}$ onto the set $\mathbb{N}^{\mathbb{N}}$ (the set of all infinite sequences of natural numbers)[17].

In computability theory it is common to construct an object without a certain effectiveness property (such as computability, computable enumerability or other) by diagonalizing against each object of the same type with this property. E.g. we can want to construct an uncomputable function (possibly additionally having some other desired properties) by diagonalizing against each computable function while ensuring that all the desired properties are maintained. To be more precise, if we construct a function g by diagonalizing against a sequence of functions $f_0, f_1, \ldots,$ we ensure that for each $i \in N$ there is an argument x such that $g(x) \neq f_n(x)$. This x is called a witness for a requirement $R_n : f_n \neq g$. Satisfying all requirements of this type ensures that function g is different from all functions f_n. In particular if (f_n) is an enumeration of all partial recursive functions, we have thus ensured that g is not recursive.

Construction of a set with requirements

This method is often used to construct a set A in infinitely many stages s indexed with subsequent natural numbers such that $A = \bigcup_{s \in \mathbb{N}} A_s$, where A_s is called aproximation of A at stage s. We aim to construct a set satisfying some requirements (R_n) (usually infinitely many and there can be various types of requirements involved in the same construction) and to do so we employ strategies for requirements. The construction describes what these strategies are and how they interact with each other. In case of constructions without injury, after a requirement is satisfied, it remains satisfied. We usually only

consider finitely many requirements at each stage of the construction. Requirements are activated at a certain stage and a corresponding strategy is employed which ensures that the requirement is eventually satisfied (e.g. due to some function returning a wrong result, thus violating an isomorphism between structures). A requirement usually has an object assigned called a witness, which guarantees that the requirement associated with that strategy is satisfied. To better illustrate this method, we will look at a simple proof.

Theorem 2.24. *There exists a c.e. set A, such that A is not computable i.e. its characteristic function χ_A is not computable.*

Proof. We construct a computably enumerable set $A \subseteq \mathbb{N}$ that is not computable by defining an infinite chain of finite approximations $A_0 \subseteq A_1 \subseteq \dots$ and letting $A = \bigcup_s A_s$.

Let $\varphi_1, \varphi_2 \dots$ be an effective enumeration of all partial computable functions $\varphi_e : \mathbb{N} \to \mathbb{N}$. We ensure that for every e, the requirement $R_e : \varphi_e \neq \chi_A$ is met. This requirement ensures that no partial computable function φ_e exists that computes χ_A. To satisfy it we use the strategy:

Strategy: Start with $A_0 = \emptyset$. At stage $s+1$, we run *dovetailing* for all $e, n \leq s$. Whenever we get $\varphi_{e,s}(n) = 1$, we prohibit n from being added to A. If $\varphi_{e,s}(n) = 0$, we enumerate n to our approximation of A more precisely, consider all $(n_i)_{i=1,\dots,k}$, such that $\varphi_{e,s} = 0$ we define $A_{s+1} = A_s \cup \{n_1, \dots, n_k\}$, otherwise do nothing.

Verification: requirement R_e clearly, guarantees that A will not be computable and it is satisfied because:

- there is no φ_e that computes χ_A, because if $\varphi_{e,s}(n) = 1$ then our strategy prohibits n from being enumerated into A, hence A is not computable. Furthermore, if for some n $\varphi_e(n) \uparrow$, then φ_e is not a total function and hence cannot be equal to χ_A. Thus we ensure that A will remain uncomputable.
- We have an effective procedure for enumerating elements of A, therefore A is *c.e* (if we want to compute if $a \in A$, we only need to wait long enough for a to show in our construction)

Therefore, we have shown that A is c.e., but not computable.

□

Finite-injury priority method

The *finite-injury priority method* was introduced independently by Friedberg [13] and Muchnik [29] in their famous solution to Post's Problem, where they constructed two c.e. sets whose Turing degrees are incomparable. This was the first demonstration that the c.e. degrees are not linearly ordered: there exist c.e. sets A and B such that $A \not\leq_T B$ and $B \not\leq_T A$.

The method works by arranging all the requirements in a priority order and allowing higher-priority requirements to override or "injure" lower-priority ones when a conflict arises. Each requirement R_e follows a strategy often with a number called witness assigned to that strategy if a higher priority requirement takes action that could interfere with lower priority requirements (e.g by enumerating an element into a set that other requirement possibly already satisfied, try to keep out of), these requirements are injured; they need to be satisfied again with new witnesses assigned. The key fact, and the reason the

construction succeeds, is that every requirement can only be injured finitely many times: each injury forces it to move to a larger witness, and higher-priority actions can only interfere finitely often below any fixed bound. Eventually, every requirement stabilizes, its restraint is preserved, and it is permanently satisfied.

We refer the reader to [25] for more in-depth description of priority arguments, an well as for modern proof of Friedberg-Muchnik theorem as an example of usage.

2.5. Group theory

In this section we will lay down standard definitions from group theory, that are necessary for the purpose of this thesis.

Definition 2.25. A tuple $\mathcal{G} = (G, \cdot^{\mathcal{G}})$, where $\cdot : G^2 \to G$, is a group if it satisfies all conditions:

- *Associativity*: for all $a, b, c \in G$, $(a \cdot b) \cdot c = a \cdot (b \cdot c)$
- *Identity element*: There exists an element in G, here denoted as e, such that for all $a \in G$, $e \cdot a = a$ and $a \cdot e = a$
- *Inverse element*: For all $a \in G$, there exists an element $b \in G$ such that $a \cdot b = e$ and $b \cdot a = e$, where e is the identity element.

Usually when writing group operation we omit the upper index in $\cdot^{\mathcal{G}}$, if it is clear from the context. Additionally, we note that depending on the nature of the operation, two common notations are used for groups. If the operation is thought of as addition, the group is called an *additive group*. The operation is denoted by $+$, the identity element is denoted by 0, and the inverse of a is written as $-a$. If the operation is thought of as multiplication, the group is called a *multiplicative group*. The operation is denoted by $\cdot$ (or often omitted), the identity element is denoted by 1, and the inverse of a is written as a^{-1}. In further definitions we will usually stick to the multiplicative notation

Remark 2.26. The identity element of a group is unique

Proof. Assume e is the identity and e' is the identity then $e = e \cdot e' = e'$. □

Remark 2.27. Every element has only one inverse

Proof. assume that a, b are both inverses of c and a, b, c are elements of the group $\mathcal{G}$. Then $a = e \cdot a = (b \cdot c) \cdot a = b \cdot (c \cdot a) = b \cdot e = b$. □

Example 2.28. The set of integers $\mathbb{Z}$ with standard addition $+$ forms a group $(\mathbb{Z}, +)$, the identity element is 0, and the inverse of a is $-a$. But $\mathbb{N}$ with standard addition does not form a group, because the standard zero would be the identity (since only for zero $n + 0 = n$). However, there does not exist $n, n' \in \mathbb{N}$, such that $n, n' = 0$ and $n + n' = 0$. Hence $(\mathbb{N}, +)$ does not form a group.

Definition 2.29. The order of a finite group is the number of its elements. If a group is not finite, one says that its order is infinite.

E.g order of the group $(\mathbb{Z}_6, +)$, where $\mathbb{Z}_6$ is integers modulo six and group operation is addition modulo 6, is equal 6, on the other hand the order of the group $(\mathbb{Z}, +)$ is infinite.

Definition 2.30. Abelian group $\mathcal{G} = (G, \cdot)$ is the group whose operation is commutative, i.e. for all $x, y \in G$, $x \cdot y = y \cdot x$

E.g. $(\mathbb{Z}, +)$ is abelian because $a + b = b + a$ for all integers a, b. As a counterexample, the group of 2×2 invertible matrices with matrix multiplication is not abelian.

Definition 2.31. Suppose that $\mathcal{G} = (G, \cdot^{\mathcal{G}})$ is a group and $H \subseteq G$. If $\mathcal{H} = (H, \cdot^{\mathcal{H}})$, where $\cdot^{\mathcal{H}} = \cdot^{\mathcal{G}} \restriction_H$, is a group, then it is called subgroup of $\mathcal{G}$. We write $\mathcal{H} \leq \mathcal{G}$ to say $\mathcal{H}$ is a subgroup of $\mathcal{G}$.

Example 2.32. In $(\mathbb{Z}, +)$, the set $2\mathbb{Z} = \{\dots, -4, -2, 0, 2, 4, \dots\}$ is a subgroup.

To verify this, we check the subgroup conditions:

- **Closure under addition:** If $a, b \in 2\mathbb{Z}$, then $a = 2m$, $b = 2n$ for some $m, n \in \mathbb{Z}$. Then $a + b = 2m + 2n = 2(m + n) \in 2\mathbb{Z}$.
- **Identity element:** The identity in $(\mathbb{Z}, +)$ is 0, and clearly $0 \in 2\mathbb{Z}$ (since $0 = 2 \cdot 0$).
- **Inverses:** If $a \in 2\mathbb{Z}$, say $a = 2m$, then its inverse is $-a = -2m$, which is also in $2\mathbb{Z}$.

Since closure, identity, and inverses hold, $2\mathbb{Z}$ is a subgroup of $(\mathbb{Z}, +)$.

Definition 2.33. Let $\mathcal{G}$ be a group. A subset $S \subseteq G$ is called a *generating set* of G if every element of G can be expressed as a finite product of elements of S and their inverses. That is,

$$G = \{s_1^{\epsilon_1} s_2^{\epsilon_2} \cdots s_n^{\epsilon_n} \mid n \in \mathbb{N},\ s_i \in S,\ \epsilon_i = \pm 1\}.$$

If such a set S exists, we say that G is *generated by* S. Group generated by set S is denoted as $\langle S \rangle$

E.g. if $S = \{(1, 0), (0, 1)\} \subseteq \mathbb{Z} \times \mathbb{Z}$, then $\langle S \rangle = (\mathbb{Z} \times \mathbb{Z}, +)$

Definition 2.34. Subgroup of a group $\mathcal{G}$ generated by an element $g \in \mathcal{G}$ is the subgroup in which every element can be expressed as a combination of g or g^{-1} and the group operation.

E.g. in $(\mathbb{Z}, +)$, the subgroup generated by 3 is $3\mathbb{Z} = \{\dots, -6, -3, 0, 3, 6, \dots\}$.

Definition 2.35. The *order of the element* in a group is the order of the subgroup generated by this element.

In other words, the order of an element of a group is the number of times we need to apply the group operation to that element to get the identity element; if we never obtain the identity element, the order is infinite. For example, the order of the subgroup $3\mathbb{Z}$, generated by 3, in $(\mathbb{Z}, +)$ is ∞, therefore the order of $3 \in (\mathbb{Z}, +)$ is infinite. On the other hand, the order of the subgroup of $(\mathbb{Z}_6, +)$, where $\mathbb{Z}_6$ means integers modulo 6, generated by 2 is 3, because $2 + 2 + 2 = 0 \pmod 6$.

Definition 2.36. *Torsion group* is a group in which every element has a finite order. This means that given a group $\mathcal{G} = (G, \cdot)$ it is true that $\forall_{g \in \mathcal{G}} \exists_{m \in \mathbb{N}} (g^m = e)$, where e is the identity element of the group $\mathcal{G}$.

For example, the group $(\mathbb{Z}_6, +)$, is torsion, because every element has a finite order.

Definition 2.37. Group in which every element has infinite order is called *torsion-free*

Definition 2.38. A *family of groups* indexed by the set I is a collection $\{G_i\}_{i\in I}$ such that for each $i \in I$, G_i is a group. That is, there exists a function

$$i \mapsto G_i$$

from the index set I to a class of groups, assigning to each $i \in I$ a group G_i.

Definition 2.39. Let $\{G_i\}_{i\in I}$ be a family of abelian groups indexed by a set I. The **direct sum** of the groups G_i, denoted by,

$$\bigoplus_{i\in I} G_i,$$

is the set of all tuples $(g_i)_{i\in I}$ such that $g_i \in G_i$ for each $i \in I$ and $g_i = 0$ for all but finitely many $i \in I$. The group operation is defined component-wise:

$$(g_i)_{i\in I} + (h_i)_{i\in I} = (g_i + h_i)_{i\in I}.$$

For example, if $I = \{1, 2\}$, $G_1 = \mathbb{Z}_2$, and $G_2 = \mathbb{Z}_3$, then $\mathbb{Z}_2 \oplus \mathbb{Z}_3$ consists of pairs (a, b) with $a \in \mathbb{Z}_2$ and $b \in \mathbb{Z}_3$, with addition defined component-wise. For instance,

$$(1, 2) + (1, 2) = (0, 1).$$

The index set I can also be infinite, but it is important that each tuple has only finitely many nonzero elements. For example, if

$$g \in \bigoplus_{i\in\mathbb{N}} \mathbb{Z},$$

then g may be of the form

$$g = (z_1, z_2, \dots, z_j, 0, 0, 0, \dots),$$

where $z_i \in \mathbb{Z}$ and after some index $j \in \mathbb{N}$, all components are zero.

Chapter 3

Important subclasses of computable structures

As stated in the introduction, our goal is to consider restricted notions of computability in relation to algebraic structures, with a particular focus on groups. Hence we aim to investigate such subclasses of computable structures that allow for more feasible computations. We focus on **punctual structures** as the most natural restriction. Nevertheless, we will also consider other subclasses of computable structures, as they have some interesting computational properties.

In the next sections, we will make use of standard asymptotic notation, the **big O notation**, for describing time complexity.

Definition 3.1. Given functions $f, g : \mathbb{N} \to \mathbb{N}$, we write:

$$f(n) = \mathcal{O}(g(n))$$

If there exist constants $c > 0$ and $n_0 \in \mathbb{N}$ such that

$$f(n) \leq c \cdot g(n) \text{ for all } n \geq n_0$$

For historical reasons, the notation $f(n) = \mathcal{O}(g(n))$ is written as an equality. However, this is somewhat misleading, since $\mathcal{O}(g(n))$ denotes a *class* (or *set*) of functions whose growth rate is bounded by $g(n)$. There are infinitely many distinct functions $f(n)$ such that $f(n) \in \mathcal{O}(g(n))$.

Definition 3.2. We say that a function (or relation) is computable in time $O(g(n))$ if there exists an algorithm that, for any input of length n, computes the function (or decides the relation) in $O(g(n))$ computation steps.

3.1. Automatic structures

Among the subclasses of computable structures we consider, automatic structures are the most restrictive. In this subsection, we review automatic structures by presenting both positive and negative results concerning which algebraic structures can be represented using finite automata.

Definition 3.3. [18] A **Deterministic Finite Automaton (DFA)** is a 5-tuple $M = (Q, \Sigma, \delta, q_0, F)$ where:

- Q is a finite set of states,

- Σ is a finite input alphabet,
- $\delta : Q \times \Sigma \to Q$ is the transition function,
- $q_0 \in Q$ is the initial state,
- $F \subseteq Q$ is the set of accepting (final) states.

Given an input string $w = a_1 a_2 \dots a_n$ where each $a_i \in \Sigma$, the DFA processes w by applying the transition function step by step: $q_1 = \delta(q_0, a_1)$, $q_2 = \delta(q_1, a_2)$, $\dots$, $q_n = \delta(q_{n-1}, a_n)$. The string is accepted if $q_n \in F$.

Automata are a very weak model of computation, however they are very feasible as they operate on bounded memory and in linear time. Below we will prove that automata compute in linear time.

Theorem 3.4. *For any input with encoding of length n, finite automata compute is the answer in time $O(n)$.*

Proof. Assume we have a string $w \in \Sigma^*$ of length n, at each step, the DFA reads a single symbol a_i from w and performs one transition. Each transition $\delta(q, a_i)$ takes same constant time, $t = \mathcal{O}(1)$. Hence, the total time required to process the input is:

$$\underbrace{t + t + \cdots + t}_{n \text{ times}} = \mathcal{O}(n)$$

Therefore, the DFA runs in **linear time** with respect to the input length. □

Definition 3.5. Algebraic structure

$$\mathcal{A} = \big(A, (R_i^{\mathcal{A}})_{i \in I}, (f_j^{\mathcal{A}})_{j \in J}\big)$$

is automatic if it satisfies the following:

- The domain A is regular set (i.e. set for which membership can be decided by DFA) of finite strings over a finite alphabet Σ, i.e., $A \subseteq \Sigma^*$ is recognized by a finite automaton.
- Each relation $R_i \subseteq A^{n_i}$ in signature is a regular relation, meaning that the set of tuples in R_i is recognized by a synchronous multi-tape finite automaton. Synchronous multi-tape means that the automaton reads one symbol from each tape simultaneously at each step (or a padding symbol if one string is shorter, denoted in this thesis as #), processes the tuple of symbols as a single input, and transitions accordingly.
- Each function $f_j : A^{m_j} \to A$ has a graph i.e

  $$\text{Graph}(f_j) = \{(x_1, \dots, x_{m_j}, y) \in A^{m_j} \times A \mid f_j(x_1, \dots, x_{m_j}) = y\}$$

  which is a regular relation, i.e., recognized by a synchronous multi-tape finite automaton.

To define synchronous reading more formally, for each i, the set:

$$R_i \subseteq A^{n_i} \subseteq (\Sigma^*)^{n_i}$$

is such that the language

$$\mathrm{c}(R_i) = \{(w_1 \otimes \cdots \otimes w_{n_i}) \mid (w_1, \ldots, w_{n_i}) \in R_i\}$$

is regular, where $\otimes$ denotes the **convolution operator** [23] of tuples of strings $(w_1, w_2, \ldots, w_n)$ that maps tuples of strings to tuples of equal length, by padding it with some symbol e.g # for synchronous reading.

Definition 3.6. If $\mathcal{B}$ is an automatic structure and $\mathcal{B} \cong \mathcal{A}$, then we say that $\mathcal{A}$ has (or admits) automatic presentation and we call $\mathcal{B}$ an automatic presentation of $\mathcal{A}$.

Example 3.7. Let $u = 10$ and $v = 110$. Then:

$$u' \otimes v' = (1,1)(0,0)(\#,1)$$

Example 3.8. Here are two simple examples of automatic structures with proofs.

- **The structure $(\mathbb{N}, S)$ has an automatic presentation.**

  Proof. **Domain:** We encode each natural number $n \in \mathbb{N}$ using unary alphabet $\Sigma = \{1\}$ as the string 1^n. The domain language $L = \{1^n \mid n \in \mathbb{N}\} \subseteq \{1\}^{<\omega}$ is clearly regular.

  Successor relation: Define the relation

  $$R_S = \{(x, y) \in L \times L \mid \overline{y} = \overline{x} + 1\}.$$

  To build a 2-tape automaton, we pad the shorter of the two strings with a special symbol # on the right, so both strings have the same length. If $x = 1^n$, then $y = 1^{n+1}$, and their padded forms become:

  $$x = 1^n\#, \quad y = 1^{n+1}.$$

  Reading these in parallel gives a sequence of symbol pairs:

  $$(1,1), (1,1), \ldots, (1,1), (\#,1).$$

  The automaton accepts only strings of this form i.e., n matching $(1,1)$ pairs followed by a single $(\#,1)$ and rejects anything else. Hence this relation is regular. □

- **The structure $(\mathbb{N}, \leq)$ has an automatic presentation.**

  Proof. Define the relation

  $$R_{\leq} = \{(x, y) \in L \times L \mid \overline{x} \leq \overline{y}\}.$$

  where $L = \{1\}^{<\omega}$ and $\bar{x}$ is the length of x. Again using right-padding with #, if $x = 1^n$, $y = 1^m$ with $n \leq m$, then their padded forms are:

  $$x = 1^n\#^{m-n}, \quad y = 1^m.$$

  This gives a tuple sequence like:

  $$(1,1), \ldots, (1,1), (\#,1), \ldots, (\#,1).$$

  The automaton accepts exactly such strings: all pairs $(1,1)$ followed by zero or more $(\#,1)$ and never any $(1,\#)$. Thus, $R_{\leq}$ is regular. □

Automatic structures are a very weak model of computation, as they do not have unbounded memory and thus are very restrictive. However, they offer some advantages in the simplicity and time complexity; they always operate in linear time with respect to the size of their input. However, as result, many algebraic structures that rely on operations requiring more than linear time computation, such as multiplication, exponentiation and those involving unbounded memory do not admit automatic structures.

As an example, if multiplication were automatic, then a finite automaton would have to read two numbers of length n and correctly compute their product, which can be as long as $2n$. But finite automata have no memory beyond a fixed number of states, so they cannot keep track of all the "carries" that happen during multiplication, since these can stretch across the whole input. In other words, multiplication creates relationships between digits that can extend indefinitely, while automata can only handle relationships that repeat in a very simple, predictable way. More formally, Khoussainov and Nerode [22] showed that multiplication fails the closure properties of automatic relations, because it requires keeping track of infinitely many different patterns of growth that no finite-state machine can handle. While this linearity makes automatic structures highly feasible computationally, it also significantly limits the class of algebraic structures that can be computed by finite automata.

Another strong property is that every automatic structure has a decidable first-order theory [23], that is if a structure has automatic presentation, then there is an algorithm that can decide whether any given first-order sentence about that structure is true or false.

Automatic presentations of finitely generated abelian groups

Finitely generated abelian groups serve as canonical examples of automatic structures [12]. In particular, free abelian groups and finite cyclic groups are known to admit automatic presentations. This includes groups such as $\mathbb{Z}^n$, $\mathbb{Z}_m$ (all finite structures have automatic copy [12]), and their finite direct products, e.g., $\mathbb{Z}^n \times \mathbb{Z}_m$.

Theorem 3.9. *[32] Every finitely generated abelian group admits an automatic presentation.*

In particular, $\mathbb{Z}^n$, $\mathbb{Z}_m$, and finite direct products of these groups have automatic presentation[6]. The intuition behind this result is that elements of such groups can be encoded using a regular, "digit-like" (e.g. binary or unary encoding) language, and the group operations correspond to addition with carry. Since addition with carry can be recognized by finite automata, this provides the foundation for automaticity.

Example 3.10. Consider the group $(\mathbb{Z}, +)$. The integers can be represented either in binary or in unary form. Finite automata are able to check addition in these encodings by maintaining a finite amount of memory to handle the carries.

Theorem 3.11. *The group $(\mathbb{Z}, +)$ admits an automatic presentation.*

Proof. We give a standard automatic presentation of $(\mathbb{Z}, +)$ using binary positional encoding with least-significant bit first (LSB-first) together with a sign marker. The proof follows two parts: regularity of the domain and recognizability of the addition relation by a synchronous finite automaton.

Domain is regular. Encode an integer $z \in \mathbb{Z}$ as follows. Write $|z|$ in binary and write the binary expansion in LSB-first order; add a sign marker $+$ or $-$ to the word. Thus the domain is

$$D = \{+w : w \in \{0,1\}^*\} \ \cup \ \{-w : w \in \{0,1\}^+\},$$

where $+w$ encodes the nonnegative integer whose LSB-first binary expansion is w, $-w$ encodes the negative of that number, and the empty word $w = \varepsilon$ represents 0 (we may choose $+\varepsilon$ for 0). The language D is regular; for example it is described by the regular expression

$$(+)|((+|-)1(0|1)^*)?$$

Addition relation on binary strings is regular. Let $x, y, z \in \mathbb{Z}$ consider their binary encoding in B, additionally define $+$ to integer addition in binary. We need to show that the ternary relation

$$\mathrm{Add} = \{(x, y, z) \in B^3 : \ x + y = z\}$$

is regular. As a slight abuser of notation, we will use the symbol $+$ for both addition of integers and for the corresponding function on strings from D, when it is clear from the context which of them we are talking about. It suffices to describe a synchronous finite automaton that reads the convolution of the three encoding and accepts exactly when the represented integers satisfy $x +^B y = z$.

Because we use LSB-first binary, addition proceeds digit-by-digit from least-significant toward more significant digits, with a carry that takes only the values 0 or 1 and in the case of different signed integers subtraction, with borrow bit that takes only the values 0 or 1. Thus a finite automaton can implement addition by storing the current carry (borrow) $c \in \{0, 1\}$ in its finite state. Concretely, at each step the automaton reads a triple of symbols (b_x, b_y, b_z) (each from $\{0, 1\}$, or a padding symbol) and checks the digitwise condition

$$b_x + b_y + c \equiv b_z \mod 2,$$

and updates the carry by $c' = \lfloor (b_x + b_y + c)/2 \rfloor \in \{0, 1\}$.

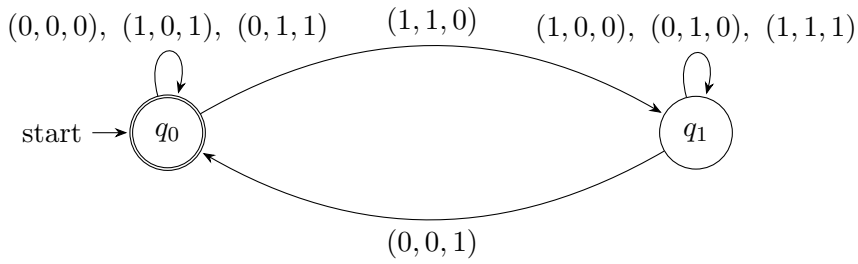
At the start the automaton is in the state carry 0. After computing all digits it accepts if the carry is 0 and it verifies of sign markers: the automaton checks the sign symbols on the three tapes and accepts only if they match the sign determined by the usual integer addition meaing:

$$x + y = \begin{cases} +(a+b) & \text{if } x = +a,\ y = +b, \\ -(a+b) & \text{if } x = -a,\ y = -b, \\ +(a-b) & \text{if } x = +a,\ y = -b \text{ and } a \geq b, \\ -(b-a) & \text{if } x = +a,\ y = -b \text{ and } b > a, \\ +(b-a) & \text{if } x = -a,\ y = +b \text{ and } b \geq a, \\ -(a-b) & \text{if } x = -a,\ y = +b \text{ and } a > b. \end{cases}$$

Therefore the ternary addition relation Add is regular. Since B is regular and Add is a regular ternary relation recognized by a synchronous finite automaton, (B, Add) *is an automatic presentation of* $(\mathbb{Z}, +)$. □

Simple automaton illustrating unsigned digitwise addition, carry machine. Below is a compact two-state automaton that illustrates the finite-state carry mechanism for addition of binary digits. The two states correspond to the current carry: q_0 represents carry 0 and q_1 represents carry 1. Each edge is labeled by the list of triples (b_x, b_y, b_z) (reading the three tapes at a single digit position) that cause the transition. The initial state is q_0 and the single accepting state is q_0 (final carry must be 0). In a full three-tape synchronous automaton, transitions would also handle padding and sign markers; this

diagram shows only the core mechanism of digitwise addition (we omit transitions that would lead to rejecting, if we can not make transition, we reject).



Each label (b_x, b_y, b_z) should be read as: when the automaton (in the given carry state) reads b_x on the x-tape, b_y on the y-tape, and b_z on the z-tape at the current digit, make the transition shown. The labels listed on an edge are exactly those digit triples for which the sum $b_x + b_y + c$ (where c is the carry in current state) yields the output bit b_z and the next carry corresponding to the target state. For example, from q_0 (carry 0), reading $(1, 1, 0)$ yields sum 2, so $b_z = 0$ and carry$c' = 1$, hence the edge $q_0 \to q_1$ is labeled $(1, 1, 0)$.

Example 3.12. Finite cyclic groups $\mathbb{Z}_m$ are automatic as well. Their elements are represented by the finite set of remainders $\{0, 1, \ldots, m-1\}$, which can be easily recognized and manipulated by a finite-state automaton.

Theorem 3.13. *If G and H are automatic groups, then their direct product $G \times H$ is automatic [18].*

This closure property ensures that direct products of copies of $\mathbb{Z}$ and $\mathbb{Z}_m$ remain automatic. The reason is that automata can simultaneously process the encodings of each component group. Formally, one constructs a product automaton whose states are the ordered pairs of the states for the individual groups.

However, to prove that a group admits an automatic presentation, it is not necessary to explicitly construct one; the class of finitely generated groups admitting automatic presentations is completely characterized by the following theorem:

Theorem 3.14. *A finitely generated group has an automatic presentation if and only if it is virtually abelian [32].*

Before proceeding, we will explain what it means for a group to be virtually abelian, since Theorem 3.14 completely characterizes the finitely generated groups that admit automatic presentations, making this concept particularly significant.

Definition 3.15. Let $\mathcal{G}$ be a group and let $\mathcal{H} \leq \mathcal{G}$ be a subgroup. For any element $g \in \mathcal{G}$, the *left coset* of H with respect to g is defined as

$$gH = \{\, gh \mid h \in H \,\}.$$

We define right coset in an analogous way. Moreover, as we can see from this definition, in the case of abelian group right and left cosets are the same.

Additionally, the set of all cosets is a quotient set, with equivalence relation being $a \sim b \iff a^{-1}b \in \mathcal{H}$. Thus we can divided domain of a group into disjoined union of cosets of this group.

Proof. We will prove that $a \sim b \iff a^{-1}b \in \mathcal{H}$ is an equivalence relation and that $[a] = \{b : a^{-1}b \in \mathcal{H}\} = \{\, a^{-1}bh \mid h \in H \,\}$, for $a, b \in \mathcal{G}$ and $\mathcal{H} \leq \mathcal{G}$

- **Reflexivity**: $a \sim a \iff a^{-1}a \in \mathcal{H} \iff e \in \mathcal{H}$, and $\mathcal{H}$ is a group.
- **Symmetry** ($a \sim b \Rightarrow b \sim a$): Assume $a \sim b$, this is equivalent to $a^{-1}b = h \in \mathcal{H}$. Therefore $b = ah \Rightarrow e = b^{-1}ah \Rightarrow b^{-1}a = h^{-1} \in \mathcal{H}$, since $\mathcal{H}$, therefore $b \sim a$.
- **Transitivity** ($a \sim b \wedge b \sim z \Rightarrow a \sim z$): Assume $a \sim b \wedge b \sim z$, this means $a^{-1}b = h$ and $b^{-1}z = h'$, where $a, b, z \in \mathcal{G}$ and $h, h' \in \mathcal{H}$. Therefore, $h^{-1}a^{-1} = b^{-1}$ and $b^{-1}z = h'$. Substituting b^{-1} to second equation we get $h^{-1}a^{-1}z = h'$. Hence, $a^{-1}z = hh' \in \mathcal{H} \iff a \sim z$.

Now we need to show that the equivalence classes of $\sim$ are exactly the left cosets of $\mathcal{H}$. Let $\mathcal{G}$ be a group and $\mathcal{H}$ be a subgroup of $\mathcal{G}$, $[a]_\sim$ denote the equivalence class of $a \in \mathcal{G}$ under the relation $\sim$. We aim to show that $[a]_\sim = a\mathcal{H}$. We proceed by showing the double inclusion.

$[a]_\sim \subseteq a\mathcal{H}$: Let x be an arbitrary element of the equivalence class $[a]_\sim$. By the definition of the equivalence class, $x \sim a$. Using the definition of the relation $\sim$:

$$\begin{aligned} x \sim a &\implies a^{-1}x = h \quad \text{for some } h \in \mathcal{H} \\ &\implies x = ah. \end{aligned}$$

Since $h \in \mathcal{H}$, it follows that $x \in a\mathcal{H}$. Therefore, $[a]_\sim \subseteq a\mathcal{H}$.

$a\mathcal{H} \subseteq [a]_\sim$: Let y be an arbitrary element of the left coset $a\mathcal{H}$. By the definition of a coset, $y = ah'$ for some $h' \in \mathcal{H}$. To determine if y is in the equivalence class $[a]_\sim$, we check if $y \sim a$:

$$\begin{aligned} a^{-1}y &= a^{-1}(ah') \\ &= (a^{-1}a)h' \\ &= eh' \\ &= h'. \end{aligned}$$

Since $h' \in \mathcal{H}$, we have $a^{-1}y \in \mathcal{H}$, which implies $y \sim a$. Therefore, $y \in [a]_\sim$, and consequently $a\mathcal{H} \subseteq [a]_\sim$.

□

Definition 3.16. A group $\mathcal{G}$ is called *virtually abelian* if it contains an abelian subgroup $\mathcal{H} \leq \mathcal{G}$ of finite index. That is:

- H is abelian: for all $h_1, h_2 \in H$, $h_1h_2 = h_2h_1$,
- with finite index $[G : H]$, i.e., the number of cosets of H in G is finite. That is, G can be written as:
$$G = g_1H \cup g_2H \cup \cdots \cup g_nH$$
for some $g_1, \ldots, g_n \in G$ and finite n. Notice that since each gH forms a an equivalence class we can partition G in such a way.

Remark 3.17. Every abelian group is virtually abelian.

Proof. Let $\mathcal{G}$ be a finitely generated abelian group. By definition, $\mathcal{G}$ is abelian, so we can take $\mathcal{H} = \mathcal{G}$ itself. Clearly, $\mathcal{H}$ is abelian, and the index $[\mathcal{G} : \mathcal{H}] = [\mathcal{G} : \mathcal{G}] = 1$, which is finite. Therefore, $\mathcal{G}$ contains an abelian subgroup of finite index (itself), so $\mathcal{G}$ is virtually abelian. □

Hence, among finitely generated groups, only those that are abelian or virtually abelian (that is, "almost abelian") admit automatic presentations. Calling a group "almost abelian" means that it may not be abelian everywhere, but the non-abelian behavior is limited. Formally, this comes from the finite index condition: the group can be split into finitely many cosets of an abelian subgroup. So while elements from different cosets may fail to commute, those "exceptions" are confined to finitely many cases, and beyond them the group's structure is controlled by the abelian subgroup. In this sense, the group looks abelian once you ignore the small, finite irregularities. However the restriction, to virtually abelian, rules out many other natural classes of finitely generated groups, such as free groups as well as all groups of infinite rank. For example:

- The free group on two generators F_2 is not abelian and it is not virtually abelian and therefore does not admit an automatic presentation [32].

  Proof. F_2 is the free group of rank two, generated by $\{a, b\}$, whose elements are reduced words in the alphabet $\{a, b, a^{-1}, b^{-1}\}$, with reduction the meaning that adjacent pairs of the form xx^{-1} or $x^{-1}x$ are cancelled. The group operation is concatenation of words, followed by reduction. It is immediate from this description that F_2 is non-abelian, and in fact more generally, every free group of rank $r \geq 2$ is non-abelian.

  A fundamental fact about free groups is given by the Nielsen–Schreier theorem [36], which asserts that every subgroup of a free group is itself free. Thus, any subgroup $H \leq F_2$ is again free, and its rank can be computed using the Schreier index formula [27]. This formula states that if F_r is a free group of rank r and $H \leq F_r$ is a subgroup of finite index n, then the rank of H is given by $\text{rank}(H) = 1 + n(r - 1)$.

  Applying this in the case $r = 2$, we see that if $H \leq F_2$ has finite index n, then $\text{rank}(H) = 1 + n \geq 2$. Therefore, every finite-index subgroup of F_2 has rank at least 2, and hence is non-abelian. This shows that F_2 is not virtually abelian. Therefore, from theorem 3.14 it follows that F_2 does not admit an automatic presentation. □

- The integer Heisenberg group $H_3(\mathbb{Z})$, i.e., the group of 3×3 upper-triangular matrices with integer entries and 1's on the diagonal, of the form $\begin{pmatrix} 1 & a & c \\ 0 & 1 & b \\ 0 & 0 & 1 \end{pmatrix}$, where $a, b, c \in \mathbb{Z}$, is not virtually abelian, and it does not admit an automatic presentation [32].

Moreover, *no infinitely generated group* can admit an automatic presentation. Intuitively, this is because automatic structures are, by definition, presented over a *finite alphabet* using finite automata that recognize the domain and the basic operations. Consequently, any structure that admits such a presentation must have a domain that can be encoded by a regular language over a finite alphabet, which corresponds to a finitely generated set of elements. Therefore, infinitely generated groups cannot be automatically presentable.

- The additive group of rationals, is not finitely generated, thus it does not admit automatic presentation [37].

- Similarly, any group isomorphic to infinite-rank abelian group $\bigoplus_{i\in\mathbb{N}}\mathbb{Z}$ (the direct sum of countably many copies of $\mathbb{Z}$) does not admit automatic presentations. However, in chapter 4, we will show that it does admit a punctual copy.

These results illustrate that automatic presentations impose significant constraints on both the algebraic structure and the generative complexity of a group. While automatic presentations offer elegant algorithmic descriptions and desirable decidability properties—such as the decidability of the first-order theory—the class of automatically presentable groups is too restrictive for many applications. Consequently, more powerful computational frameworks are needed to represent and analyze more complex algebraic structures, such as *punctual structures* or polynomial-time structures.

For example, although not all torsion-free abelian groups admit automatic presentations, it has been shown that they can all be represented using **punctual presentations**, as will be detailed in Section 4.

Further reading: For a more in-depth treatment of automatic abelian groups and the surrounding theory, the reader is referred to [12, 31, 7, 30]. These sources explore both *geometric* and automata theoretic perspectives.

3.2. PTIME structures

Definition 3.18. A **polynomial-time structure** over a finite signature is an algebraic structure

$$\mathcal{A} = \big(A,\ (R_i^{\mathcal{A}})_{i\in I},\ (f_j^{\mathcal{A}})_{j\in J},\ (c_\ell^{\mathcal{A}})_{\ell\in L}\big)$$

such that:

- The domain A can be encoded as a subset of Σ^* (strings over a finite alphabet) whose membership can be decided in polynomial time.
- Each basic relation $R_i \subseteq A^{n_i}$ has a polynomial-time decision procedure (i.e., its characteristic function is in P).
- Each basic function $f_j : A^{m_j} \to A$ can be computed in polynomial time on a multi-tape Turing machine or an equivalent model of computation.

We say that a set, relation or function is computable in polynomial time if there is a polynomial $p(n)$ such that, given any input tuple of total length n, the algorithm for R_i or f_j returns the correct answer in at most $p(n)$ steps.

Discussion on the domain of PTIME structures

When studying computable structures, we usually take the domain to be ω (or a subset of it), and the particular choice of encoding does not play a significant role. In contrast, for automatic structures, only certain presentations of the domain are regular, and for polynomial-time (PTIME) structures, the situation is even more delicate because, as we can see from Definition 3.18, the encoding matters. As Cenzer and Remmel [9] point out, not all infinite polynomial-time sets are polynomial-time isomorphic (that is, their isomorphism need not be computable in polynomial time). Moreover, the choice between binary and unary encodings leads to different behaviors.

3.2.1. Examples of algebraic structures that admit PTIME copy

Definition 3.19. A structure $\mathcal{A}$ is said to *admit a PTIME presentation* (or equivalently, to have a *PTIME copy*) if there exists a PTIME structure $\mathcal{B}$ such that $\mathcal{A} \cong \mathcal{B}$. That is, $\mathcal{A}$ is isomorphic to some structure $\mathcal{B}$ whose domain, functions, and relations are all computable in polynomial time.

Cenzer, Remmel, Downey, and their collaborators developed an approach to computational feasibility grounded in polynomial-time structures. The study of polynomial-time algebras has become a well-established area, with numerous significant results (see, e.g., [9, 10, 8, 15]). However, as noted in [20], much of this work does not directly address the interaction between computable structure theory and polynomial-time algebra. For this reason, we restrict attention here to a few key results from [10, 9], which illustrate the connections between PTIME and the feasible computation of algebraic structures.

We first focus on some examples of PTIME structures that are not necessarily groups. Remmel and Cenzer began the study of computable structures that are isomorphic to PTIME structures in their 1991 paper [9]. Here are positive results from this paper:

- *Relational Structures*: Every computable relational structure (i.e., with no function symbols) is computably isomorphic to a polynomial-time structure.
- *Boolean Algebras*: Every computable Boolean algebra can likewise be represented as a polynomial-time Boolean algebra.
- *Arithmetic on the Natural Numbers*: The computable structure $(\mathbb{N}, S, +, \times, 2^n, <)$ is computably isomorphic to a polynomial-time structure.

Already we can see the connection between PTIME structures and fully primitive recursive ones, as every computable *Boolean algebra* not only has a PTIME presentation but also has a fully primitive recursive one (punctual) [20]; the same, of course, applies to arithmetic on the natural numbers. Moreover, as noted by Alaev [20], every relational structure with a finite alphabet language admits a primitive recursive copy (however, not a punctual copy, as we will see in Section 3.3). Further in this thesis, we will see that PTIME does not guarantee punctuality (see Theorem 4.7 [10] combined with Theorem 4.13).

Examples of structures that do not admit polynomial presentation

Cenzer and Remmel also showed some negative results on the PTIME presentability of structures:

- *Unary Function Structures*: There exist recursive structures with a single unary function that are not recursively isomorphic to any polynomial-time structure.
- *Multiple Function Symbols*: With two unary functions or one binary function, recursive structures can be constructed that fail to be polynomial-time isomorphic even under more general notions of isomorphism.
- *Universe Restrictions*: PTIME presentability is strongly depened on the choice of the universe. For example if one fixes the universe of a PTIME structure to be $0, 1^*$, then there exists a computable linear ordering of order type $\omega + \omega^*$ that fails to have PTIME-isomorphic copies.

These counterexamples show that complexity constraints impose real structural differences that cannot always be bridged by isomorphisms.

3.2.2. Comparison with automatic structures

PTIME structures are a significantly broader class than automatic structures. Recall from the section 3.1 that automatic structures are those whose domain and relations can be recognized by finite automata, and hence their operations are constrained to what can be computed in linear time with strictly bounded memory. In contrast, PTIME structures allow any polynomial-time algorithm to compute the relations of the structure. This difference greatly enlarges the class of representable structures. In particular, since PTIME structures are characterized by being computable in $O(n^k)$ time and automatic structures are always $O(n)$, the class of automatic structures is a subset of PTIME structures.

For instance, multiplication of two binary integers of length n can be carried out in $O(n^2)$ time using the standard schoolbook algorithm (see the example below), or even faster using more sophisticated methods. This means that $(\mathbb{N}, \times)$ is PTIME-presentable. On the other hand, as we already know, multiplication is not automatic.

Example 3.20. Multiplication of two n-bit binary integers can be computed in $O(n^2)$ time.

Proof. Let $A = a_{n-1} \dots a_0$ and $B = b_{n-1} \dots b_0$, so that $A = \sum_{i=0}^{n-1} a_i 2^i$ and $B = \sum_{j=0}^{n-1} b_j 2^j$.

For each bit of B, we add a shifted copy of A into the result. Each addition takes $O(n)$ time, and we perform at most n of them, giving a total time of $O(n^2)$. □

Thus, the schoolbook multiplication algorithm uses $O(n^2)$ bit-level operations, so its running time is $O(n^2)$.
Similarly, consider languages of arithmetic with order. Automatic presentations can handle $(\mathbb{N}, +, <)$ (as we saw in the previous section), because addition and comparison can be computed by finite automata using binary representations. However, more complex predicates/operations such as primality testing, finite automata no longer suffice. Primality testing, is not automatic (finite automata cannot verify it), but since it can be checked in polynomial time (e.g., using the AKS primality test), the predicate $\mathsf{Prime}(n)$ can be incorporated into a PTIME-presentable structure.

These simple examples already show that PTIME structures allow us to compute more complex structures.

3.2.3. PTIME groups

Cenzer and Remmel proved a strong negative result, demonstrating that not all computable groups admit a PTIME copy.

Theorem 3.21. *There exists a computable abelian group $\mathcal{G}$ that is not computably isomorphic to any primitive recursive abelian group.*

The proof of this Theorem uses a diagonal method (see section 2.4), by building a computable abelian group, while diagonalizing against all possible pairs $(\mathcal{A}_\rangle, \varphi_i)$ of PTIME structures and computable isomorphisms, ensuring that resulting structure is exponential-time.

The addition operation of $\mathcal{G}$ is defined to depend on a function that grows faster than any fixed polynomial. Consequently, any attempt to interpret the group operation within a PTIME bound fails to preserve the algebraic structure under computable isomorphism.

Characterization of groups that admit PTIME copy

Using results from the section on automatic structures, we can conclude that any group admitting an automatic presentation necessarily admits a PTIME presentation, since every automaton operates in linear time. In particular, as a consequence of Theorems 3.9 and 3.14, we obtain the following results:

Theorem 3.22. *Every finitely generated abelian group admits a PTIME presentation.*

Theorem 3.23. *If a finitely generated group $\mathcal{G}$ is virtually abelian, then $\mathcal{G}$ admits a PTIME presentation.*

These results are highly encouraging for the study of PTIME presentability among abelian groups. Additionally, P. E. Alaev in [1] showed strong result:

Theorem 3.24. *If $\mathcal{A} = (A, \cdot^{\mathcal{A}})$ is a group computable in polynomial time, then there exists a P-computable group $\mathcal{B} = (B, \cdot^{\mathcal{B}}) \cong \mathcal{A}$ in which the inverse group operation is also computable in polynomial time.*

In [10] Cenzer and Remmel constructed an abelian group of finite order that is not computably isomorphic to a polynomial-time group with the universe $Tal(\omega)$ or $Bin(\omega)$(i.e strings of ones representing natural numbers or binary representation), showing once again that PTIME structures are sensitive to representation of their domain.

In the same paper they provide an even stronger result, substantially extending the class of known PTIME presentable abelian groups. They establish the following theorem:

Theorem 3.25. *If every element of a computable abelian group has finite order, then the group is computably isomorphic to a polynomial-time group.*

Consequently, even abelian groups that are not finitely generated have a PTIME copy, provided that all of their elements generate finite subgroups. In particular, all torsion abelian groups admit a PTIME copy. However, as we demonstrate in Chapter 4, there exists a *torsion abelian group that does not admit a punctual presentation.*

This observation is especially noteworthy in light of the work of [20], where it is noted that, in many cases, PTIME presentability is established by first proving that a structure is primitive recursive. Yet, as we see here, certain structures can be PTIME without being fully primitive recursive.

3.3. Punctual structures

In this section we will lay down definitions that will be necessary for the central part of this thesis namely Chapter 4.

Definition 3.26 ([20])**.** An algebraic structure is *primitive recursive* if its domain is a primitive recursive (p.r) subset of ω, and its operations and relations are (uniformly) primitive recursive.

Notice that this notion of **uniformly primitive recursive functions**, operations, and relations is more ambiguous than the corresponding notion of uniformly computable functions (see Definition 2.19). To illustrate this ambiguity, consider a structure $\mathcal{A} = (A, (f_i)_{i \in I})$, where I is a set of indices for a not necessarily finite family of functions, and suppose we attempt to define a primitive recursive candidate algorithm $p(i, \bar{x})$, analogous to φ in Definition 2.19. In order to compute $p(i, \bar{x})$ we would need to know the arity of the

function f_i, which may vary with i. This arity could in principle be supplied by a primitive recursive function returning the arity of f_i, by a primitive recursive relation encoding arities, or perhaps only by a computable function. However, the original definition in [20] does not specify how the arities are to be provided, leaving the uniformity requirement somewhat unclear. Since in our setting we deal exclusively with groups, whose signatures contain only a single binary operation, this issue does not arise for us.

If our goal is to impose meaningful restrictions on computable structures, Definition 3.26 does not adequately constrain the domain:

"as observed by Alaev (personal communication), every computable structure whose finitely generated substructures are finite has a primitive recursive copy... they (p.r structures) seem too close to (general) computable structures to be a good intermediate notion."[20]

The reason is that, when constructing a primitive recursive copy from a computable presentation, we can delay introducing elements into its domain $D \subseteq \omega$ until sufficient amount of the corresponding original computable presentation has been observed. Below we formalize and prove this

Theorem 3.27. *All computable relational structures over a finite signature admits a primitive recursive copy*

Proof. Let $\mathcal{A} = (A, R_1^{\mathcal{A}}, R_2^{\mathcal{A}}, \ldots, R_k^{\mathcal{A}})$ be computable relational structure over a finite signature (no function symbols). Then $A \subseteq \omega$ is computable and each R_i is computable as well. We will build an isomorphic structure $\mathcal{B}$ with a primitive recursive domain $B \subseteq \omega$ and all $R_i^{\mathcal{B}}$ primitive recursive. We will carry out the construction of $\mathcal{B}$ in stages $s \in \omega$, enumerating $\mathcal{A}$, but delaying adding numbers to B until enough of $\mathcal{A}$ is shown.

Construction: At the beginning of stage $s = 0$, $\mathcal{B} = \emptyset$ and we have not computed any part of $\mathcal{A}$.

At stage s: we have a computable approximation of $\mathcal{A}$, denoted as the $\mathcal{A}_s$, and we compute finite relational structure induced on A_s.

We also have an isomorphism $f_s : A_s \to B_s \subseteq \omega$, where B_s is the domain constructed up to stage s. For all relations R_i in the signature and all $a_1, \ldots, a_n \in \mathcal{A}_s$, where n is the arity [1] of each relation R_i, we compute whether $R_i(a_1 \ldots, a_n)$ is true (we can ignore questions that have already been computed for better time complexity).

Let k be number be the current step of construction (not to be confused with stage, every step consists of a single action), if $s \in \mathcal{A}_s$, then assign $\psi(s) = k$.

Membership in the new domain B is primitive recursive, since a number n can only be added to B after k steps of construction and we only add one number at each stage.

$$n \in B \iff \exists k \leq n \text{ such that } n \text{ is placed into } B \text{ at step } k,$$

which is a bounded search.

It remains to show that each relation $R_i^{\mathcal{B}}(b_1, \ldots, b_n)$ in the new structure is primitive recursive. To compute $R_i^{\mathcal{B}}$, we carry out k steps of construction where $k = max(b_1, \ldots, b_n)$, compute $a_1 = \psi^{-1}(b_1), \ldots, a_n = \psi^{-1}(b_n)$, check if $R_i(a_1, \ldots, a_n)$ and return the same answer. Since we never change any decisions once made, and since a tuple involving only

[1] Without a loss of generality we can assume that all relations R_i have the same arity

numbers $b_1, \ldots, b_n$ can only be decided by the step $k = max(b_1, \ldots, b_n)$, the question can be resolved by a bounded search. Therefore, for any relation R in the singature,

$$(x_1, \ldots, x_m) \in R^{\mathcal{B}} \iff \exists (x_1, \ldots, x_m) \text{ has been enumerated into } R^{\mathcal{B}} \text{ in some step } k \leq max(\{x_1, \ldots, x_m$$

and this is primitive recursive. Hence the resulting structure $\mathcal{B}$ is a primitive recursive copy of the original computable structure.

□

As we can see, by manipulating the domain we are able to construct a p.r copy of any computable relation over a finite signature. This shows that the restriction to primitive recursive subset of ω is too relaxed, giving us too general class of structures that is very close too the class of computable structures. This is because the method used in the proof of Theorem 3.27 encodes the difficulty of a structure (the necessary delay) in its domain rather than in the relations from the signature. To counter this weakness of primitive recursive structures, a stronger requirement can be imposed on the domain.

Definition 3.28. [20] A countable structure is *fully primitive recursive (punctual, f.p.r)* if its domain is ω or some initial segment of it and all of functions and relations from signature are (uniformly) primitive recursive.

The same comment about uniformity as in the case of Definition 3.26 applies here, we recall that we are only concerned with structures over finite signature in this thesis. Additionally, since finite object over finite signature are trivial from perspective of computability and primitive recursive, we are going to ignore finite cases in proofs and consider only structures with infinite domains.

The Definition 3.28 does not permit the kind of delay we employed in the proof of Theorem 3.27. There, we could simply wait for a sufficiently large substructure to appear and then enumerate a new element into a primitive recursive copy. In contrast, for a punctual copy, after a predictable number of steps a new part of the structure must be added to the copy in order for the domain to be ω.

From Theorem 3.31, item (3), we know that there exists a computable undirected graph that does not admit a punctual copy. From this fact, together with the proof above, we see that there are structures that have a primitive recursive copy but no fully primitive recursive copy, since undirected graphs are relational structures over a finite signature.

3.3.1. Punctually robust classes of structures

A natural question arieses to ask: *which classes of computable structures admit punctual copies?*

Definition 3.29. A class of structures is called *punctually robust* if every computable member of that class admits a punctual presentation.

This definition was originally introduced in [21]. Note that it does not require the isomorphism between the computable structure and the punctual copy to be primitive recursive. Therefore, when constructing a punctual copy, we often just need to define the operations, relations, and domain of the copy before even knowing their counterparts in the original computable structure. As we will see in the proof of Theorem 4.6, this greatly relaxes the constraints by allowing us to predefine natural numbers in the copy, thereby avoiding the delay caused by computing preimages in the original structure.

Positive and negative results about punctually robust structures. Below we list some classes of structures known to be punctually robust.

Theorem 3.30. *[20] The following classes of structures are punctually robust:*

1. *equivalence structures,*
2. *linear orders,*
3. *torsion-free abelian groups,*
4. *abelian p-groups.*
5. *Boolean algebras,*

Theorem 3.31. *The following classes of structures are not punctually robust:*

1. *torsion abelian groups,*
2. *Archimedean ordered abelian groups,*
3. *undirected graphs,*
4. *structures with only injective function in the signature,*
5. *trees as partial orders,*
6. *join semilattices and meet semilattices,*
7. *lattices, complemented lattices, and non-distributive lattices, both under order-theoretic and algebraic interpretation,*
8. *modal algebras.*

The results 1-3 come from [20] and 4-5 from [21] and 6-8 from [5].

From those results we can infer a useful heuristic: a class of structures is likely to be punctually robust when

1. during construction of punctual copy, we can quickly enumerate a certain infinite subset of the original structure without needing to quickly define isomorphism on those elements.

2. elements can be introduced independently, without requiring prior knowledge about yet-unrevealed parts of the structure.

This intuition has been formalized by Alaev in Theorem 4 in [2], where he characterized conditions for the existence of punctual copy.

Chapter 4

Punctual presentability of torsion and torsion-free abelian groups

In this main chapter of the thesis we will focus on two results about punctual presentability of abelian groups, namely torsion abelian groups and torsion-free abelian groups. Original proofs for both those classes of groups come form [20], proofs presented here are more detailed and in the case of proof 4.13 corrected versions of the originals.

To prove these theorems, we need additional definitions and theorems from computable structure theory and group theory.

4.1. Torsion-free abelian groups

Firstly, for the proof of Theorem 4.6, we introduce the definition of linearly independent set and related definition *basis*, which differs from the usual definition in group theory. This version will be more convenient for the purpose of proof of theorem 4.6.

Definition 4.1. Let $\mathcal{G} = (G, +)$ be a group with identity element 0 the set $A \subseteq G$ is linearly independent if for all $a_1, \ldots, a_k \in A$ and all $n_1, \ldots, n_k \in \mathbb{Z}$,

$$n_1 a_1 + n_2 a_2 + \cdots + n_k a_k = 0 \implies n_1 = n_2 = \cdots = n_k = 0$$

. Otherwise we say that A is linearly dependent.

Definition 4.2. A *basis* of a group is a maximal linearly independent set of group elements, i.e. adding any new element would make the set linearly dependent.

Additionally, for the proof of Theorem 4.6, it will be useful to define free abelian groups and demonstrate their relation to torsion-free abelian groups.

Definition 4.3. A group $\mathcal{G}$ is called a *free abelian group* if there exists a subset $S \subseteq G$ such that every element $g \in \mathcal{G}$ can be written uniquely as

$$g = n_1 s_1 + n_2 s_2 + \cdots + n_k s_k,$$

where $s_i \in S$ and $n_i \in \mathbb{Z}$. The set S is called a *generating set* of G, and the cardinality of S is called the *rank* of $\mathcal{G}$.

Let us also remark that, by definition, every finitely generated abelian group is free abelian; in particular, every finitely generated torsion-free abelian group is free abelian. However, not every torsion-free abelian group is free abelian.

Lemma 4.4. *Not every torsion-free abelian group is free abelian.*

Proof. We will prove this by constructing a torsion-free abelian group $\mathcal{G}$ that is not free abelian. First, consider the group

$$\mathbb{Z}^s = \underbrace{\mathbb{Z} \oplus \mathbb{Z} \oplus \cdots \oplus \mathbb{Z}}_{s \text{ times}} = \prod_s \mathbb{Z}.$$

of infinite sequences of integers, with the group operation being component-wise addition. This group has a generating set

$$B = \{b_1, \dots, b_s\},$$

such that for each $i = 1, \dots, s$.

$$b_i = (b_{1,i} \dots, b_{s,i})$$

where $b_{i,i} = 1$ and $b_{i,j} = 0$ for all $i \neq j$. For any $a = (a_1, \dots, a_s) \in \mathbb{Z}^s$, if $a_i \neq 0$ for some $1 \leq i \leq s$, then b_i is needed to generate a. Now consider the group

$$\mathbb{Z}^\omega = \prod_{s \in \omega} \mathbb{Z},$$

with standard component-wise addition. By the argument above, for such a group $|B| = \infty$. Additionally, consider $a = (a_i)_{i \in \omega} \in \mathbb{Z}^\omega$ such that $a_i \neq 0$ for infinitely many i. Then, by the same argument,

$$a = \sum_{i \in \omega} n_i b_i, \quad \text{where each } a_i = n_i b_i.$$

If $\mathbb{Z}^\omega$ were free abelian with some generating set B, then by definition every element of $\mathbb{Z}^\omega$ could be expressed as a finite $\mathbb{Z}$-linear combination of elements of B. Consider the element $a = (1, 1, 1, \dots)$. Since a has infinitely many nonzero coordinates, it cannot be written as a finite linear combination of elements of B, which is a contradiction.

□

Below we present the proof of Theorem 1.5 from [20]. In our reconstruction we fill in the gaps in the original proof which was quite vague in some parts. For the purpose of this proof, first we will prove the following Lemma:

Lemma 4.5. *Let $A = \{a_1, a_2, \dots, a_n\}$ be a basis set of abelian group $\mathcal{G}$, $\mathcal{H}$ be an group isomorphic to $\mathcal{G}$ and $\psi : H \to G$ be an isomorphism and each b_i is an isomorphic preiamge of $a_i \in A$, then set $\{b_1, \dots, b_n\}$ is a basis of $\mathcal{H}$*

Proof. Assume we have a basis set $A = \{a_1, a_2, \dots, a_n\}$ of the the abelian group $\mathcal{G}$, a group $\mathcal{H}$ an isomorphic to $\mathcal{G}$ and isomorphism $\psi : H \to G$, where each b_i is an isomorphic preiamge of $a_i \in A$. For all $n_1, n_2, \dots, n_i \in \mathbb{Z}$:

$$n_1 a_1 + n_2 a_2 + \cdots + n_i a_i = n_1 \psi(b_1) + n_2 \psi(b_2) + \cdots + n_i \psi(b_i) = \psi(n_1 b_1 + n_2 b_2 + \cdots + n_i b_i)$$

Assume $n_1 a_1 + n_2 a_2 + \cdots + n_i a_i = 0$ and $\{a_1, a_2, \dots, a_i\}$ is linearly independent, meaning

$$n_1 a_1 + n_2 a_2 + \cdots + n_i a_i = 0 \implies n_1 = n_2 = \cdots = n_i = 0$$

Assume $n_1 b_1 + \cdots + n_i b_i = 0$ then

$$\psi(n_1 b_1 + n_2 b_2 + \cdots + n_i b_i) = \psi(0) = 0 = n_1 a_1 + n_2 a_2 + \cdots + n_i a_i$$

and since ψ is partial injective isomorphism, for the first equality to hold the arguments of ψ must be equal, hence we get

$$n_1b_1 + n_2b_2 + \cdots + n_ib_i = 0 \iff 0 = n_1a_1 + n_2a_2 + \cdots + n_ia_i$$

Therefore

$$n_1b_1 + n_2b_2 + \cdots + n_ib_i = 0 \implies n_1 = n_2 = \cdots = n_i = 0$$

Hence, $\{b_1, b_2, \ldots, b_i\}$ is linearly independent □

4.1.1. Proof: The class of torsion-free abelian groups is punctually robust

Theorem 4.6. *The class of torsion-free abelian groups is punctually robust.*

Proof. Let $\mathcal{G}$ be a computable torsion-free group. We construct a punctual group $\mathcal{H} \cong \mathcal{G}$ and a computable isomorphism $\phi : \mathcal{H} \to \mathcal{G}$. The construction is carried out in infinitely many stages $s = 0, 1, 2, \ldots$, indexed with natural numbers.

First, we need to establish the following facts:

1. Every computable torsion-free abelian abelian group has a computable copy with a computable basis [16]. Therefore, in our proof, we will assume we are working with a presentation of $\mathcal{G}$ that has a computable basis (in the sense of Definition 4.2).

2. A finitely generated subgroup of a torsion-free abelian group, by definition, is free abelian.

3. Every generating set of the group can be replaced by a linearly independent one. Thus, we assume the generating set is linearly independent.

We construct $\mathcal{G}$ of an infinite rank (i.e. infinite cardinality of the basis set), since this case is more complex; the finite case is just an easy modification, where the basis extension strategy is removed.

Let

$(a_i)_{i\in\omega}$ be a computable basis of the computable torsion-free abelian group $\mathcal{G}$.

In this proof we will construct finite approximations of infinite objects, indexed with subscripts for stage of construction (e.g $\mathcal{H}_s$). We will sometimes omit these subscripts if they are clrear from the context.

At stage $s = 0$ we have $\mathcal{H} = \emptyset$ and have not yet computed any of the basis of $\mathcal{G}$.

At the beginning of stage $s > 0$:

- We have enumerated elements into $\mathcal{H}_s$ with basis set $\{b_1, \ldots, b_s\}$ and a generating set $\{e_1, \ldots, e_s\}$.

- Note that $\mathcal{H}_s$ is a partial group, i.e. the group operation may not yet be defined for all elements.

- We have an approximation of $\mathcal{G}$, denoted $\mathcal{G}_s$.

- We have defined a partial embedding, i.e. an injective map $\psi_s : \mathcal{H}_s \to \mathcal{G}_s$, such that $\psi_s(b_i) = a_i$, where $i \leq s$, and range$(\psi_s) \subseteq \mathcal{G}_s$. Note that this embedding is not necessarily primitive recursive because we must wait for more of $\mathcal{G}$ to be revealed before defining a_i, and $\mathcal{G}$ is only computable.

Since at stage s every b_i is an isomorphic preimage of a_i, $\psi_s(b_i) = a_i$ we know, by Lemma 4.5, that $b_1, b_2, \ldots, b_i$ is linearly independent, additionally because every b_i is a preimage of unique a_i, the set $\{b_1, b_2, \ldots, b_i\}$ forms a basis of $\mathcal{H}_s$. Hence, every $h \in \mathcal{H}_s$ satisfies

$$\exists k \in \mathbb{Z} \text{ such that } k \cdot h = \sum_{i \leq s} n_i b_i.$$

This is because, even if h is not a linear combination of the basis elements $b_1, \ldots, b_s$, some multiple of it is. Suppose, for contradiction, that there is no $k \neq 0$ such that kh is a linear combination of $b_1, \ldots, b_s$. Then $b_1, \ldots, b_s, h$ would be linearly independent, contradicting the fact that $b_1, \ldots, b_s$ is a maximal linearly independent set. Therefore, such k exists.

The partial group $\mathcal{H}_s$ has a generating set $e_1, \ldots, e_s$ and is defined as

$$\mathcal{H}_s = \langle e_1 \rangle_m \oplus \cdots \oplus \langle e_s \rangle_m,$$

where each $\langle e_i \rangle_m$ is a partial finite group generated by e_i, defined as

$$\{-me_i, \ldots, -e_i, 0, e_i, \ldots, me_i\}.$$

We increase m at each step of the construction using the **copy-delay strategy**. Gradually increasing m at each stage ensures punctuality of $\mathcal{H}$ (see Lemmas 4.7, 4.8). Note that at each stage, $\mathcal{H}_s$ is a partial (i.e the group operation is not necessarily defined on all group elements) free abelian, since it is a finitely generated torsion-free group. However, it is not necessarily free abelian in the limit as $s \to \infty$ (see Corollary 4.4).

Construction:

In the construction, we use three strategies. When strategies "enumerate" elements into $\mathcal{H}$, they assign least unused natural number to every element generated by the current generating set, effectively listing all integer linear combinations $n_1e_1 + n_2e_2 + \cdots + n_se_s$.

- **Copy-delay strategy.** This strategy ensures that $\mathcal{H}$ is punctual. We increase m by some fixed constant value and grow $\mathcal{H}_s$ by enumerating all group elements of the form $h = n_1e_1 + \cdots + n_se_s$, where each $n_i \in [-m, m]$.

- **Basis-extension strategy.** This strategy ensures punctuality in the case of infinite rank of the group. We extend the basis with new elements b_{s+1} while waiting for the a_{s+1} appear in $\mathcal{G}_s$, to set $\psi_s(b_{s+1}) = a_{s+1}$. We enumerate the least not yet used natural number into $\mathcal{H}_s$ as the new element and define it as b_{s+1}. We first extend $\mathcal{H}_s$ to $\mathcal{H}_s \oplus \langle b_{s+1} \rangle$ and enumerate all needed the elements into $\mathcal{H}_{s+1}$, setting $b_{s+1} = e_{s+1}$,

- **Copy-onto strategy.** This strategy ensures ψ is an isomorphism from $\mathcal{G}$ to $\mathcal{H}$. It waits until computation (that is always working in the "background") of $\mathcal{G}$ finds some $g \in \mathcal{G}$, then it takes g as a witness and for enough of the basis of $\mathcal{G}$ to appear so that for some fixed k:

  $$kg = \sum_{i \leq s} n_i a_i.$$

  We describe this in more detail. Suppose at current stage we have already enumerated $a_1, \ldots, a_j$ into basis of $\mathcal{G}$, for all $k \leq s$ and all $n_i \in [-m, m]$ (m is defined by construction), we check wether the above is true. Then we check whether the current $\mathcal{H}_s$ will ever have an element h such that for each a_i and the above fixed $k, n_1, \ldots, n_i$:

  $$kh = \sum_{i \leq s} n_i b_i.$$

To perform this check, we replace each b_i with a linear combination of elements of the generating set $b_i = \sum_j r_{j_i} e_{j_i}$ and check whether k divides every $n_i r_{j_i}$. This procedure is computable.

If such h exists, we wait for this h to appear in the approximation of $\mathcal{H}$ and declare $\psi(h) = g$. If for some $g \in \mathcal{G}$ there is no $h \in \mathcal{H}_s$ satisfying

$$kh = \sum_{i \leq s} n_i b_i,$$

we expand the group by a new generator e_{s+1} such that $ke_{s+1} = \sum_{i \leq s} n_i b_i$. However, this would make our generating set linearly dependent and have greater cardinality than the basis set. To avoid this we replace one element e_i of generating set with h (we can always generate this element by linear combination of h and other elements of generating set, since the previous set is linearly dependent). Finally, we set

$$\mathcal{H}_{s+1} = \mathcal{H}_s \oplus \langle e_{s+1} \rangle.$$

In other words we define a new generating set $e_1, \ldots, e_{s+1}$ for $\mathcal{H}_{s+1}$ so that one of the generators (say e_{s+1}) is equal to h. Since $\mathcal{H}_{s+1}$ is free abelian, this is always possible, and all previously defined basis elements and values of ψ remain unchanged. Thus, the isomorphism is preserved, and the construction continues uniformly.

The Copy-delay and Basis-extension strategies work one after another at each stage, while we wait for the Copy-onto strategy to finish its computation. Thus, we ensure that the construction remains primitive recursive, even if $\mathcal{G}$ and ψ are only computable. Additionally, we define $\psi(b_s)$ only after its intended image $a_s \in \mathcal{G}_s$ appears, ensuring that $\mathcal{H}$ remains punctual and ψ remains computable.

Combining these strategies, we construct a computable group $\mathcal{H}$ that is punctual, torsion-free abelian, and computably isomorphic to $\mathcal{G}$.

Verification:

Lemma 4.7. *The domain of $\mathcal{H}$ is ω.*

Proof. By construction, at every stage $s > 0$ we *enumerate at least one new natural number* into $\mathcal{H}_s$, this follows from the copy-delay and basis-extension strategies: each stage either extends the generating set by a new generator or enumerates further finite linear combinations in all cases. Hence during each stage at least one previously unused natural number is enumerated into the domain of $\mathcal{H}$. Hence, after s stages

$$\{0, 1, \ldots, s\} \subseteq dom(\mathcal{H}_s)$$

and

$$\bigcup_{s \in \omega} \{0, 1, \ldots, s\} = \omega \subseteq \bigcup_{s \in \omega} dom(\mathcal{H}_s) = dom(\mathcal{H})$$

since $dom(\mathcal{H})$ is countable and $\omega \subseteq dom(\mathcal{H})$, we know that the domaine of $\mathcal{H}$ is precisely omega ω.

Hence, the domain of $\mathcal{H}$ is ω. □

Lemma 4.8. *The group operation of $\mathcal{H}$ is primitive recursive.*

Proof. Since the copy-delay and basis-extension strategies extend $\mathcal{H}_s$ at each stage s, we can guarantee that if $h', h'' \in \mathcal{H}$, then both elements will appear in some stage $\mathcal{H}_s$ (by the same argument as in Lemma 4.7 in stage $s \leq max(h', h'')$). Once h' and h'' are found (primitive recursively), we can also find the corresponding generating set $e_1, \ldots, e_s$ of $\mathcal{H}_s$.

$$h' = n'_1 e_1 + \cdots + n'_s e_s, \qquad h'' = n''_1 e_1 + \cdots + n''_s e_s.$$

In the construction, the copy-delay strategy increases the bound m by a fixed value (for example, $m_{s+1} = m_s + 1$). Because this increase is primitive recursive, we can effectively determine the stage at which all coefficients $n'_i + n''_i$ lie within $[-m, m]$. At that stage, the element

$$h = h' + h'' = (n'_1 + n''_1)e_1 + \cdots + (n'_s + n''_s)e_s$$

appears in $\mathcal{H}_s$. Since the length of each stage of this computation depends only on effective, bounded searches, the operation $(h', h'') \mapsto h' + h''$ is primitive recursive. □

Lemma 4.9. *ψ is a computable isomorphism.*

Proof. We assume $\psi(b_i) = a_i$, i.e. ψ maps the basis of $\mathcal{H}$ to the basis of $\mathcal{G}$.

- **Computability:** By construction we define ψ effectively whenever we declare a preimage in the copy-onto step, so ψ is computable.
- **Homomorphism:** Let $h, h' \in \mathcal{H}$. Choose positive integers k, k' and representations.

  $$kh = \sum_i n_i b_i, \qquad k'h' = \sum_i n'_i b_i.$$

  We want to show that $\psi(h + h') = \psi(h) + \psi(h')$. Let $M = \mathrm{lcm}(k, k')$, then

  $$M(h + h') = Mh + Mh' = \sum_i M\frac{n_i}{k} b_i + \sum_i M\frac{n'_i}{k'} b_i = \sum_i M\Big(\frac{n_i}{k} + \frac{n'_i}{k'}\Big) b_i.$$

  Applying ψ and using $\psi(b_i) = a_i$ we get

  $$\psi\big(M(h + h')\big) = \sum_i M\Big(\frac{n_i}{k} + \frac{n'_i}{k'}\Big) a_i = \psi(Mh) + \psi(Mh').$$

  Hence

  $$M\big(\psi(h + h') - \psi(h) - \psi(h')\big) = 0.$$

  Since $\mathcal{G}$ is torsion-free, we can cancel M and obtain

  $$\psi(h + h') = \psi(h) + \psi(h'),$$

  so ψ is a homomorphism.
- **Injectivity:** Suppose $\psi(h) = \psi(h')$. Then $\psi(h - h') = 0$. By Copy-onto strategy for some k

  $$k(h - h') = \sum_i m_i b_i.$$

  applying ψ yields

  $$0 = \psi\big(k(h - h')\big) = \sum_i m_i a_i.$$

  The (a_i) are linearly independent, so each $m_i = 0$, hence $k(h - h') = 0$. Because $\mathcal{H}$ is torsion-free, $h - h' = 0$, i.e. $h = h'$. Hence ψ is injective.

- **Surjectivity** By the copy-onto strategy, for every $g \in \mathcal{G}$ the construction finds an $h \in \mathcal{H}$ with $\psi(h) = g$. Hence ψ is surjective.

Hence, ψ is a computable bijective homomorphism, i.e. a computable isomorphism. Therefore, for any torsion-free $\mathcal{A}$ group there exists punctual copy $\mathcal{A}'$ of $\mathcal{A}$ and a computable isomorphism $\psi : A \to A'$

□

□

Corollary 4.10. *The inverse operation of $\mathcal{H}$ is primitive recursive.*

Proof. In the copy-delay strategy, we extend the group by increasing m in $h = n_1e_1 + \cdots + n_se_s$, where $n_i \in [-m, m]$. Therefore, we add the inverse of any element along with this element at the same stage (to find the inverse of h, we simply switch each n_i to $-n_i$ and compute appropriate sum). □

4.2. Torsion abelian groups

Unlike the class of torsion-free abelian groups, the class of torsion abelian groups is not punctually robust. Below we provide a proof of this fact, adapted from [20] (Theorem 3.2 in the original source). The original proof contained an error in the formulation of the auxiliary strategy: we intended to enumerate new primes into the set S infinitely many times, but the lower bound on the primes being added to S (to be exact $\sqrt[m]{m}$, where m was upper bound on the order) tended to 1 as the construction progressed. We have corrected this issue and rewritten the proof in greater detail.

First for we prove the following lemma, that will be useful in our proof:

Lemma 4.11. *If $P = \{q_1, q_2, \cdots, q_n\}$ is a finite set of prime numbers, then the group $G = \bigoplus_{q_i \in P} \mathbb{Z}_{q_i}$ is isomorphic to group $\mathbb{Z}_{q_1 \cdot q_2 \cdot \ldots \cdot q_n}$.*

Proof by induction. We will prove the base of induction in a more general setting, where q_1 and q_2 are a pair of coprimes.

Base of induction: Let q_1, q_2 be a pair of coprime numbers, then the following is true $\mathbb{Z}_{q_1} \times \mathbb{Z}_{q_2} \cong \mathbb{Z}_{q_1 \cdot q_2}$ Without a loss of generality we can treat a group of the form $\mathbb{Z}_n$ as the *quotient set* of equivalence classes $\mathbb{Z}_n = \{[x]_n : x \in \mathbb{Z}\}$, where $[x]_n = \{y \in \mathbb{Z} : y \equiv x \pmod{n}\}$ and addition in $\mathbb{Z}_{q_1} \times \mathbb{Z}_{q_2}$ is coordinate-wise. We define the isomorphism $\psi : \mathbb{Z}_{q_1 \cdot q_2} \to \mathbb{Z}_{q_1} \times \mathbb{Z}_{q_2}$, as follows:

$$\psi([x]_{q_1 \cdot q_2}) = ([x]_{q_1}, [x]_{q_2}).$$

1. ψ is injective:

$$\psi([x]_{q_1 \cdot q_2}) = \psi([y]_{q_1 \cdot q_2}) \implies [x]_{q_1 \cdot q_2} = [y]_{q_1 \cdot q_2}$$

From the antecedent, we have:

$$[x]_{q_1} = [y]_{q_1} \text{ and } [x]_{q_2} = [y]_{q_2}$$

$$x \equiv y \mod q_1$$

$$x \equiv y \mod q_2$$

From this, we conclude that $q_1 \mid (x-y)$ and $q_2 \mid (x-y)$. Since q_1 and q_2 are coprime, it follows that:

$$q_1 \cdot q_2 | (x-y)$$

and this is equivalent to:

$$x \equiv y \mod q_1 \cdot q_2.$$

.

2. ψ is surjective: we will construct an inverse function, such function exists, because ψ is injective:

$$\psi^{-1}([a]_{q_1}, [b]_{q_2}) = [x]_{q_1 \cdot q_2}.$$

We construct a function that returns the value of x satisfying the congruences:

$$x \equiv a \mod q_1$$

$$x \equiv b \mod q_2$$

By the Chinese remainder theorem [14], if q_1, q_2 are coprimes, there exists a unique value of $x \in \mathbb{Z}_{q_1 \cdot q_2}$ satisfying both congruences, and it can be easily calculated. Hence, ψ is surjective.

3. ψ is a homomorphism:

$$\psi([x+y]_{q_1 \cdot q_2}) = ([x+y]_{q_1}, [x+y]_{q_2}).$$

Form the definition of addition in product of groups it follows that:

$$([x+y]_{q_1}, [x+y]_{q_2}) = ([x]_{q_1}, [x]_{q_2}) + ([y]_{q_1}, [y]_{q_2}).$$

Using the definition of ψ we get:

$$([x]_{q_1}, [x]_{q_2}) + ([y]_{q_1}, [y]_{q_2}) = \psi([x]_{q_1 \cdot q_2}) + \psi([y]_{q_1 \cdot q_2}).$$

Therefore, ψ is an isomorphism and it is true that $\mathbb{Z}q_1 \times \mathbb{Z}q_2 \cong \mathbb{Z}_{q_1 \cdot q_2}$.

Having proven the base case (any two prime numbers are always coprimes) we assume that $\mathcal{G} = \bigoplus_{q_i \in P} \mathbb{Z}_{q_i} \cong \mathbb{Z}_{q_1 \cdot q_2 \cdot \ldots \cdot q_n}$ for set of n prime numbers, and prove it for set of $n+1$ prime numbers.

If $\{q_1, q_2, \ldots, q_n\}$ is set of primes and q_{n+1} is prime then $q_1 \cdot q_2 \cdot \ldots \cdot q_n$ and q_{n+1} are coprimes. It follows from the base case that

$$\mathbb{Z}_{q_1 \cdot q_2 \cdot \ldots \cdot q_n} \times \mathbb{Z}_{q_{n+1}} \cong \mathbb{Z}_{q_1 \cdot q_2 \cdot \ldots \cdot q_n \cdot q_{n+1}}$$

. This concludes the proof of the inductive case.

□

Corollary 4.12. *For every $q_1, q_2, \ldots, q_n \in P$ there are computable presentations $\mathcal{A}, \mathcal{B}$ of $\mathbb{Z}_{p_1 \cdot p_2 \cdot \ldots \cdot p_n}$ and $\bigoplus_{i=1}^{n} \mathbb{Z}_{p_i}$ respectively such that the isomorphism $\psi : A \to B$ is computable and so is ψ^{-1} .*

Proof. We explicitly describe computable procedures for ψ.

ψ: Given an input x with $0 \leq x < p_1 p_2 \ldots p_n$, for each i we compute the remainder

$$a_i = x \bmod p_i.$$

Computing $x \bmod p_i$ is a basic arithmetic operation, and performing this for finitely many i yields

$$(a_1, a_2, \ldots, a_n).$$

Thus, ψ is computable and by Lemma 4.11 is bijection, therefore its inverse ψ^{-1} is computable.

□

4.2.1. Proof: The class of torsion abelian groups is not punctually robust

Theorem 4.13. *The class of torsion abelian groups is not punctually robust*

Proof. We construct a computable torsion group:

$$\mathcal{A}_S = \bigoplus_{p \in S} \mathbb{Z}_p$$

where S is an infinite set of primes, $\mathcal{A}_S$ is a direct sum (Definition 2.39) of groups $\mathbb{Z}_{p_i}$, such that $\mathcal{A}_S$ is not isomorphic to any punctual structure.

Notice that $\mathcal{A}_S$ has a computable copy if and only if S is computably enumerable.

Lemma 4.14. *$\mathcal{A}_s$ has computable copy $\iff$ S is computably enumerable.*

Proof. We prove both implication:
($\Leftarrow$) S is c.e. $\Rightarrow$ $\mathcal{A}_S$ has computable copy
We will construct a computable copy of $\mathcal{A}_S$ in stages. Since S is c.e., assume that at the beginning of stage $s > 0$ we have enumerated primes

$$q_1, q_2, \ldots, q_e,$$

where each $q_i \in S_s$ and $e \leq s$. We also have a computable copy of

$$\mathcal{A}_{S_s} = \bigoplus_{q \in S_s} \mathbb{Z}_q.$$

When a new prime q_i is enumerated into S, we extend our group to

$$\mathcal{A}_{S_{s+1}} = \bigoplus_{q \in S_{s+1}} \mathbb{Z}_q,$$

where $S_{s+1} = S_s \cup \{q_i\}$.

For each new summand $\mathbb{Z}_{q_i}$ we allocate q_i fresh natural numbers and encode its elements as pairs (i, r), where i is the index of the prime q_i and $r < q_i$ is the residue. We identify each such pair with a natural number using a fixed computable encoding (e.g. Cantor pairing) and add it to the domain. Addition within each component is performed modulo q_i, and addition between elements from different components acts component-wise, leaving coordinates not mentioned unchanged.

Since the enumeration of S is computable, and at each stage the operation on new elements is defined by a total computable rule, the limit structure has a computable domain and a total computable group operation. Therefore $\mathcal{A}_S$ has a computable copy.

($\Rightarrow$) $\mathcal{A}_s$ has a computable copy $\Rightarrow$ S is c.e

Assume that $\mathcal{A}_S$ has a computable copy $\mathcal{B}$. We now construct a computable enumeration of S in stages. At stage $s = 0$, let $S_0 = \emptyset$. For each subsequent stage $s > 0$, suppose we have already enumerated s elements $a_1, \ldots, a_s$ of $\mathcal{A}_S$. For each a_i, compute

$$s \cdot a_i = \underbrace{a_i + a_i + \cdots + a_i}_{s \text{ times}}.$$

If for some a_i we find that $s \cdot a_i = 0$, then then order of a_i divides s. In that case, we enumerate all prime divisors of s into the set S.

Proceeding this way, we obtain an effective enumeration of S. To verify correctness, fix a prime q. If $q \in S$, then there exists a nonzero element $a_q \in \mathcal{B}$ of order p. Hence, at some stage some $s \geq q$, we will have $q \cdot a_p = 0$, and hence q will be enumerated into S. Conversely, if $q \notin S$, then no element of $\mathcal{A}_S$ has order q, so q will never be enumerated. Therefore, S is computably enumerable.

□

Having proved the Lemma, we can now prove the main Theorem:

Construction of $\mathcal{A}_s$

First, we notice that thanks to Lemma 4.14, to construct $\mathcal{A}_s$ that is computable, but not isomorphic to any punctual structure, we can construct such a c.e set of primes S that $\mathcal{A}_s \not\cong (\omega, +_e)$ for every binary primitive recursive function p_e. To emphasize that p_e is a binary operation we will denote it as $+_e$ instead of p_e.

We will construct the set S in infinitly many stages s indexed with natural numbers. Using Lemma 4.11, without a loss of generality, at each of stage of the construction we can treat the group $\bigoplus_{q \in S_s} \mathbb{Z}_q$ as the group $\mathbb{Z}_P$, where $P_s = \prod_{q_i \in S_s} q_i$ and S_s is the current approximation of set the S at the beginning of stages s.

We have an effective enumeration of punctual structures

$$(\omega, +_1), (\omega, +_2), \ldots, (\omega, +_i)$$

where $+_e$ is a p.r function indexed with e, we are going to write $\mathcal{B}_e = (\omega, +_e)$. Such enumeration exists, because the domain of each structure is simply ω and because there exists an effective enumeration of all primitive recursive functions (see Theorem 2.14). Each structure $(\omega, +_e)$ has a requirement associated with it:

$$R_e : (\omega, +_e) \not\cong \mathcal{A}_S$$

Whenever we try to diagonalise against (an isomorphism type of) some structure, we will first treat this structure as if it was a torsion abelian group. If at some point it is discovered not to be one (e.g. $a +_e b = a +_e c$ holds for some $b \neq c$ and hence this is not a group at all), then we automatically ensure satisfaction of the corresponding requirement (the structure in question is guaranteed not to be isomorphic to the torsion group under construction). In game-theoretic terms we can think of each requirement as an opponent and then discovering that $(\omega, +_e)$ is not a torsion group qualifies as a win against opponent R_e.

It can also be the case that some part of e-th structure will look like a torsion-free group in the limit. This will happen if the infinite sequence $a, a +_e a, a +_e a +_e a, a +_e a +_e a +_e a, \ldots$ does not contain any repeating elements. In this case we win even if we do not become aware of that after any finite number of steps.

The most tricky case is when, after revealing a finite (possibly large) part of the opponent's structure, it still looks like a torsion group. As long as this is the case, we will try to ensure our win even in the worst case scenario when none of the easy win conditions described above work. While treating the structure as a potential torsion group, we try to determine ranks of its elements and to ensure that they are not the same as ranks of elements in the computable group under construction. Note that the set of ranks of elements in groups is preserved under isomorphisms.

To be more precise, we want to check for what $k \in \mathbb{N} \setminus 0$ it holds that[1]

$$k \cdot_e a = \underbrace{a +_e \cdots +_e a}_{k \text{ times}} = 0.$$

Unfortunately, even assuming that our opponent's structure is a group, we do not know what number stands for 0 (the neutral element) in that group. We try to "guess" it in the following way: we continue building a sequence $a, 2\cdot_e, 3 \cdot_e a, ...$ until for some $k > 1$ we discover that $k \cdot_e a = a$. Then we "guess" that $k - 1$ is the rank of element a and that $(k-1) \cdot_e a$ is this group's zero element.

If repeating the same procedure on another number yields a different candidate for the zero element, then our opponent's structure is not a group and we win against him.

For each $e \in \mathbb{N}$ the construction needs to ensure that:

- either there is an element $a \neq 0^{\mathcal{A}_S}$ such that $\langle a \rangle$ is a group of infinite order under $+_e$ (and a subgroup of $\mathcal{A}_S$).
- or there is an element $a \neq 0^{\mathcal{A}_S}$ such that $\langle a \rangle$ is a finite torsion group under $+_e$ (and a subgroup of $\mathcal{A}_S$), furthermore if k is the order of that subgroup, i.e. $k \cdot a = 0^{\mathcal{A}_S}$, then the construction must guarantee that at least one prime divisor of k is not in S.

We arrange requirements in the standard (numerically by indices) priority order by

$$R_0 > R_1 > R_2 > \dots$$

During our construction, at each stage s, we will a build computable approximation of S.

$$S_0 \subseteq S_1 \subseteq \cdots \subseteq S = \bigcup_s S_s$$

Additionally each requirement R_e has two associated strategies: the main strategy and the auxiliary strategy arranged in the same priority order. Each main strategy tries to satisfy R_e and each auxiliary strategy tries to enumerate a new prime into S, to ensure that S is infinite.

Stage 0 We start with $S_0 = \emptyset$.

Stage $s > 0$. At stage s we have a finite set of primes S_s, which is the current approximation of S. We process each requirement R_e for $e \leq s$ in order of priority, except for those that have been declared satisfied and for each active requirement we have associated the main and the auxiliary strategy. For each active requirement and each number $a \leq s$ we first try to implement its main strategy (prohibit some number) and if we are currently

[1] Observe that in this notation the first factor is interpreted as the standard natural number and only the second one as a member of $\mathcal{B}_e$.

unable to do that, we implement the auxiliary strategy (enumerate some number). We repeat this process for every $a = 0, ..., s$ and then we move to the next requirement and its strategy until we visit every active requirement. Below we describe in more detail how we handle each active requirement R_e at stage s for some number a.

Fix R_e and a. We simulate the first m steps additions of a under $+_e$ in $(\omega, +_e)$ in the following sense:

we check, for all $k \leq m$, whether $k \cdot_e a = 0^{\mathcal{B}_e}$, where $k \cdot_e a = \underbrace{a +_e \cdots +_e a}_{k \text{ times}}$. he parameter m controls how many times we are going to iterate addition of a under $+_e$ and will be increases every time a new number is enumerated into S. This parameter is computed using the following formula:

$$m = q' \prod_{q \in S_s} q,$$

where q' is the least prime, that is not already in S_s and has not been prohibited by a higher-priority strategy or its associated main strategy in a previous stage.

The **main strategy** will act according to either of two cases described below, depending on the outcome of $k \cdot_e a$. Also, depending on the outcome of the main strategy, we may also implement the **auxiliary strategy** described later.

and an **auxiliary strategy** described below.

Main strategy:

- **Case 1:** $(k \cdot_e a \neq 0^{\mathcal{B}_e})$ If we compute that, for all $1 < k \leq m$, $k \cdot_e a_e \neq 0^{\mathcal{B}_e}$, we proceed to the auxiliary strategy.
- **Case 2:** $(k \cdot_e a = 0^{\mathcal{B}_e})$ If we find some $a \leq m$ and $k \leq m$, such that $k \cdot_e a = 0^{\mathcal{B}_e}$, we prohibit l - a prime factor of k from being enumerated into S, and declare R_e to be satisfied and inactive. More specifically, we choose l to be the least prime divisor of k that is not in the current approximation of S. In particular, if l is already prohibited (or if k has any prohibited prime divisor at all), no new ban needs to be introduced. If at some later point a higher-priority strategy enumerates l into S, then R_e will be injured and will become active again. If at some later point higher-priority strategy enumerates l into S, then R_e will become active again. However, a requirement whose auxiliary strategy already managed to enumerate a number into S, never needs to do that again, even if injured.

Auxiliary strategy: For each R_e, its auxiliary strategy tries to enumerate the least prime q', that is not already in S_s and has not been prohibited by a higher-priority strategy or its associated main strategy in a previous stage.

To check whether q' can be enumerated into S_s, the auxiliary strategy computes

$$m = q' \prod_{q \in S_s} q,$$

(the same parameter as defined earlier). Note that some strategy auxiliary strategy of some requirement $R_i, i < e$ could have already enumerated an element into S_s at stage s, therefore m can be different for each R_i and can change during stage s. Hence, it needs to be recalculated whenever a new number is added to S.

If, for all $i \leq e$ and $a \leq m$, $m \cdot_i a \neq 0^{\mathcal{B}_e}$, then the auxiliary strategy of R_e is allowed to enumerate q' into S_s.

At the end of stage s, we set $S_{s+1} = S_s \cup \{q' : R_e \text{ was allowed to enumerate } q'\}$.

Verification

Lemma 4.15. *S is an infinite computably enumerable set of primes.*

Proof. By construction, every auxiliary of some requirement R_e, has to check if enumerating a new prime would not result in finite order for any higher-priority requirement. However, since each $+_e$ is primitive recursive this check is done in finite time, additionally R_e can be injured by adding new prime number to S only $e+1$ times. This is because each requirement $R_0, ..., R_{e-1}$ can only add at most one prime to S. Even injuring some R_i does not force its auxiliary strategy to add another prime again later. For $i \geq e$, no R_i can add a number to S that would injure (the result of the main strategy of) R_e. Therefore, since there are infinitely many strategies R_e, each will eventually enumerate different primes after finitely many stages. Hence, S is an infinite set of primes. Additionally once a prime has been enumerated into S, there is no way for it to be removed, therefore S is c.e. □

Lemma 4.16. *$\mathcal{A}_S = \bigoplus_{p \in S} \mathbb{Z}_p$ has a computable copy.*

Proof. This immediately follows from Lemmas 4.14, 4.15. □

Lemma 4.17. *$\mathcal{A}_S$ is not isomorphic to any punctual structure.*

Proof. According to Lemma 4.11, to ensure that no $(\omega, +_e)$ is isomorphic to $\mathcal{A}_S$ at any stage s, we guarantee that if for some $a \in \omega$, $k \cdot_e a = a$, $(\omega, +_e)$ does not contain an element whose order equals a finite product of some primes in S_s. At each stage s, we have three possible outcomes depending on the order k of an element $a \in (\omega, +_e)$.

Case 1: $k \cdot_e a = 0^{\mathcal{B}_e}$.

- If there exists a prime factor of k that has not yet been enumerated into S by higher-priority strategies, we prohibit this prime factor from being enumerated into S. Thus, we ensure that k cannot be equal to any multiple of $\prod_{q \in S_s} q$. Hence, $(\omega, +_e) \not\cong \mathcal{A}_S$.

- If all the prime factors of k have already been enumerated into S by higher-priority strategies (injury), we wait until some later stage $s' > s$ when some element a with a new order $k' \neq k$ is discovered such that k' has different prime factors than k. We then prohibit one of these new prime factors, ensuring that $(\omega, +_e) \not\cong \mathcal{A}_S$.

  If such a k' never appears at any later stage $s' > s$, then R_e is satisfied in the limit, since S is constructed to be an infinite set of primes, while the orders of elements of $(\omega, +_e)$ only has have finitely many distinct prime factors. Hence $(\omega, +_e) \not\cong \mathcal{A}_S$.

Case 2: $k \cdot a \neq 0$. In this case, we never find an element a such that $k \cdot_e a = 0$. Thus, the order of a is infinite, and $(\omega, +_e)$ is clearly not isomorphic to any torsion group. □

Therefore, $\mathcal{A}_s$ has a computable copy but it does not admit punctual copy, hence the class of torsion abelian groups is not punctually robust.

□

Chapter 5

Conclusions

The purpose of this thesis was to investigate punctual presentations of abelian groups, computable presentations whose operations and relations are primitive recursive, while the domain is either $\mathbb{N}$ or a finite initial segment there of, we situated them within the broader context of subclasses of computable structures. Punctual structures form a natural and robust subclass: they are expressive enough to capture a wide range of algebraic objects, yet restrictive enough to guarantee totality, boundedness, and predictability of computation. This balance makes them a compelling framework for studying effective algebra.

The main results of the thesis concern the punctual presentability of two important subclasses of abelian groups: torsion-free groups (Theorem 4.6) and torsion groups (Theorem 4.13). For both classes, we present expanded and more detailed versions of the original proofs (see [20]), and in the torsion case we correct a mistake that rendered the original construction invalid. In the torsion-free case, we show that every computable torsion-free abelian group admits a punctual presentation. The proof proceeds by constructing the group operation in a controlled way that ensures all required computations can be carried out using primitive recursion. In other words, the torsion-freeness aligns naturally with the bounded growth behavior characteristic of primitive recursive computation.

In contrast, the situation for torsion groups is fundamentally different. We prove that not every computable torsion abelian group has a punctual presentation. The problem arises because torsion abelian group can encode unbounded finite searches in the order of its elements, and certain computable torsion groups effectively require computational resources beyond primitive recursion. This separation illustrates a natural boundary in the theory of punctual structures: while they are expressive, they do not encompass all computable presentations. We also correct an error present in the original proof of this result.

In addition to this, we place punctual structures in context by comparing them with other resource bounded classes such as automatic and PTIME structures. Automatic structures, though computationally very efficient, are too restrictive to model many algebraic operations. PTIME structures offer greater expressive power but are defined through a complexity-theoretic lens rather than a direct restriction on recursion and may seem as an arbitrary restriction. Thus, punctual structures occupy a conceptual middle ground: they are vastly broader than automatic structures while maintaining uniform totality and avoiding unbounded computation.

Future Directions

Another natural direction is to continue the study of punctually robust classes by seeking additional positive and negative results that extend those already given in Theorems 3.31 and 3.30.

In recent paper [2] P.Alaev proposed some new results about punctual presentability of groups and he formulated the following question:

Is it true that every computable abelian group $\mathcal{A} = (A, +, -,)$ has a primitive recursive presentation?

While as show in the Theorem 4.13 answer to similar question (but posed for fully primitive recursive presentation) is negative, the solution to Aleav's problem to the best of our knowledge remains unknown

Bibliography

[1] P. E. Alaev. The complexity of inversion in groups. *Algebra and Logic*, 62(2):103–118, 2023.

[2] Pavel E. Alaev. The existence of primitive recursive structures. In Arnold Beckmann, Isabel Oitavem, and Florin Manea, editors, *Crossroads of Computability and Logic: Insights, Inspirations, and Innovations*, pages 119–132, Cham, 2025. Springer Nature Switzerland.

[3] Henk P Barendregt. Introduction to lambda calculus. 1984.

[4] Nikolay Bazhenov, Rod Downey, Iskander Kalimullin, and Alexander Melnikov. Foundations of online structure theory. *Bulletin of Symbolic Logic*, 25(2):141–181, 2019.

[5] Nikolay Bazhenov, Dariusz Kalociński, and Michał Wrocławski. Online and feasible presentability: from trees to modal algebras. *arXiv preprint arXiv:2504.16663*, 2025.

[6] Till Blumensath. *Automatic Structures*. PhD thesis, RWTH Aachen, 1999.

[7] Gábor Braun and Lutz Strüngmann. Breaking up finite automata presentable torsion-free abelian groups. *International Journal of Algebra and Computation*, 21(08):1463–1472, 2011.

[8] Douglas Cenzer, Rodney G Downey, Jeffrey B Remmel, and Zia Uddin. Space complexity of abelian groups. *Archive for Mathematical Logic*, 48(1):115–140, 2009.

[9] Douglas Cenzer and Jeffrey Remmel. Polynomial-time versus recursive models. *Annals of Pure and Applied Logic*, 54(1):17–58, 1991.

[10] Douglas Cenzer and Jeffrey Remmel. Polynomial-time abelian groups. *Annals of Pure and Applied Logic*, 56(1-3):313–363, 1992.

[11] Alan Cobham. The intrinsic computational difficulty of functions. In Yehoshua Bar-Hillel, editor, *Logic, methodology and philosophy of science*, pages 24–30. North-Holland Pub. Co., 1965.

[12] David B. A. Epstein, M. S. Paterson, J. W. Cannon, D. F. Holt, S. V. Levy, and W. P. Thurston. *Word Processing in Groups*. A. K. Peters, Ltd., USA, 1992.

[13] Richard M. Friedberg. Two recursively enumerable sets of incomparable degrees of unsolvability (solution of post's problem, 1944). *Proceedings of the National Academy of Sciences of the United States of America*, 43(2):236–238, 1957.

[14] Carl Friedrich Gauss. *Disquisitiones arithmeticae*, volume 157. Yale University Press, 1966.

[15] Serge Grigorieff. Every recursive linear ordering has a copy in dtime-space (n, log (n)). *The Journal of Symbolic Logic*, 55(1):260–276, 1990.

[16] Matthew Harrison-Trainor, Alexander Melnikov, and Antonio Montalbán. Independence in computable algebra. *Journal of Algebra*, 443:441–468, 2015.

[17] Felix Hausdorff. *Set theory*, volume 119. American Mathematical Soc., 2021.

[18] John E Hopcroft, Rajeev Motwani, and Jeffrey D Ullman. Introduction to automata theory, languages, and computation. *Acm Sigact News*, 32(1), 2001.

[19] Edmonds Jack. Paths, trees, and flowers. *Canadian Journal of Mathematics*, 17:449–467, 1965.

[20] Iskander Kalimullin, Alexander Melnikov, and Keng Meng Ng. Algebraic structures computable without delay. *Theoretical Computer Science*, 674:73–98, 2017.

[21] Dariusz Kalociński, Luca San Mauro, and Michał Wrocławski. Punctual presentability in certain classes of algebraic structures. In *49th International Symposium on Mathematical Foundations of Computer Science (MFCS 2024)*, pages 65–1. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2024.

[22] Bakhadyr Khoussainov and Anil Nerode. Automatic presentations of structures. In Daniel Leivant, editor, *Logic and Computational Complexity*, pages 367–392, Berlin, Heidelberg, 1995. Springer Berlin Heidelberg.

[23] Bakhadyr Khoussainov and Anil Nerode. *Automata Theory and its Applications*. 2001.

[24] Stephen Cole Kleene. General recursive functions of natural numbers. *Mathematische annalen*, 112(1):727–742, 1936.

[25] Steffen Lempp. Priority arguments in computability theory, model theory, and complexity theory. *Lecture notes*, 2012.

[26] A. Mal'cev. Constructive algebras. *Uspekhi Matematicheskikh Nauk*, 16(3(99)):3–60, 1961.

[27] J. Peter May. *A concise course in algebraic topology*. University of Chicago press, 1999.

[28] Albert R Meyer and Dennis M Ritchie. The complexity of loop programs. In *Proceedings of the 1967 22nd national conference*, pages 465–469, 1967.

[29] Albert Abramovich Muchnik. Negative answer to the problem of reducibility of the theory of algorithms. *Doklady Akademii Nauk SSSR*, 108(2):194–197, 1956.

[30] André Nies and Pavel Semukhin. Finite automata presentable abelian groups. In Sergei N. Artemov and Anil Nerode, editors, *Logical Foundations of Computer Science*, pages 422–436, Berlin, Heidelberg, 2007. Springer Berlin Heidelberg.

[31] André Nies and Richard M. Thomas. Fa-presentable groups and rings. *Journal of Algebra*, 320(2):569–585, 2008. Computational Algebra.

[32] Graham P. Oliver and Richard M. Thomas. Automatic presentations for finitely generated groups. In *Proceedings of the 22nd Annual Conference on Theoretical Aspects of Computer Science*, STACS'05, page 693–704, Berlin, Heidelberg, 2005. Springer-Verlag.

[33] Michael O. Rabin. Computable algebra, general theory and theory of computable fields. *Transactions of the American Mathematical Society*, 95(2):341–360, 1960.

[34] Hartley Rogers Jr. *Theory of recursive functions and effective computability.* MIT press, 1987.

[35] Richard Shore and Cornell Univeristy. Computable structures: Presentations matter. 2000.

[36] John Stillwell. *Classical topology and combinatorial group theory*, volume 72. Springer Science & Business Media, 2012.

[37] Todor Tsankov. The additive group of the rationals does not have an automatic presentation. *The Journal of Symbolic Logic*, 76(4):1341–1351, 2011.